

Diffusion, Flow Matching, Distillation & Real-Time Inference

A Practitioner's Handbook

July 2026

How to read this document

Part I — Foundations

- The probabilistic core (no calculus)
- Where flow matching sits
- The geometry: why paths curve
- The two sources of curvature
- Terminal SNR: a bug you can see
- SDE, ODE, and why deterministic sampling is valid
- Naming: flow matching, rectified flow, stochastic interpolants
- What space does diffusion run in?
- The DiT backbone

Part II — Training recipe (flow matching)

- Why flow matching, honestly ranked
- The complete training loop
- Timestep distributions and why they are bell-shaped
- Resolution shift — the single most common serious bug
- Classifier-free guidance and condition dropout
- Conditioning a *frozen* model: ControlNet and the adapter family
- Multi-resolution training
- Training checklist, ranked by impact

Part III — Sampling and solvers

- Do people use solvers? Always
- The error formula: everything follows from this
- Why Euler is the right choice for flow matching
- Solver selection table
- Step schedules
- SDE vs ODE at sampling time

Part IV — Distillation recipe

- When to distil
- Why determinism is required
- The three families
- Why a GAN loss is safe here
- The full family map — and which ones you actually need
- Guidance distillation (a separate 2×)
- Inference after distillation — no solver
- Reflow, and why it lost

Part V — Inference optimisation recipe

- The mental model: memory-bound vs compute-bound
- The complete map: ten categories of inference optimisation
- Kernels, fusion, and launches
- Token count and the quadratic trap
- Quantisation
- torch.compile vs TensorRT vs ONNX
- Profiling
- The ordered optimisation recipe

Part VI — Real-time video

- Why offline models cannot stream
- Drift — the central research problem
- Hardware co-design
- Kernel engineering is hardware-specific
- Case study: Decart
- Build plan: converting a batch model to streaming

Part VII — Interactive world models

- What makes a world model different
- The two axes, and why conflating them wastes months
- MIRA — joint generation, and diffusing in representation space
- MultiGen — put the world outside the network
- Matrix-Game 3.0 — learned retrieval memory
- The world-model landscape

Part VIII — Evaluation & metrics

- What each metric actually catches
- Validate by relative ordering, not absolute FID
- The three evaluations people skip

Part IX — Quick reference

- Model landscape
- Decision tables
- Speedup ledger
- Formulas
- Glossary
- Reading list
- Ecosystem: when HuggingFace, when raw PyTorch
- Cost reference

Part X — Project notes

- Findings in the vendored DIAMOND code
- Open item in our own package
- Open questions
- Corrections log

The whole document in twelve lines

How to read this document

This handbook is organised so that the **recipes** (Parts II–V) can be used standalone, while Part I supplies the reasoning behind every choice they make.

Part	Content	Use it when
I	Foundations — the probabilistic core, geometry, ODE/SDE, latent space , the DiT backbone	You want to know <i>why</i> a recipe says what it says
II	Training recipe (flow matching), conditioning & adapters	You are training a model
III	Sampling & solvers	You are choosing a sampler / step count
IV	Distillation recipe	You need fewer than ~15 steps
V	Inference optimisation recipe	You need lower latency
VI	Real-time video & drift	You need streaming / sub-100 ms
VII	Interactive world models	You are building something playable
VIII	Evaluation & metrics	You need to prove a change helped
IX	Quick reference, ecosystem , costs	Day-to-day lookup
X	Project notes	Working in <i>this</i> repo

Three rules govern nearly everything in here:

1. **Fewer steps beats faster steps.** Step-count reduction is a $25\times$ lever. Kernel work is a $1.1\text{--}2\times$ lever. Never invert the order.
2. **Curvature $\times$ step size = error.** Every solver, path, and schedule decision is an attack on one of those two factors.
3. **Modern GPUs compute far faster than they can fetch.** Almost all low-level optimisation is about moving fewer bytes.

Two more that govern Parts VI–VIII specifically:

4. **A generative model that consumes its own output has a feedback problem, not a quality problem.** Drift is structural; you fix it in the training loop, never in the sampler.
5. **Every scalar quality metric hides at least one failure mode.** Report a fidelity number and a diversity number, or you are not measuring.

Part I — Foundations

The probabilistic core (no calculus)

The problem

You have samples from an unknown distribution $p_{\text{data}}(x)$ — real images. You want new samples. But p_{data} in pixel space is a fantastically complicated, razor-thin distribution folded through millions of dimensions. You cannot write it down, normalise it, or sample from it directly.

The trick: build a ladder of easier distributions

Do not model p_{data} . Instead **blur it** by increasing amounts. Define q_t as the distribution obtained by taking a real image and adding Gaussian noise of width σ_t :

$$q_t(x) = \int N(x; x_0, \sigma_t^2) \cdot p_{\text{data}}(x_0) dx_0$$

That is simply **convolution**. q_t is a blurred copy of p_{data} . Look at the two ends:

$\sigma = 0$	→ $q = p_{\text{data}}$	impossibly complicated
σ small	→ p_{data} , slightly smeared	still complicated
σ medium	→ modes merging	
σ LARGE	→ $q \approx N(0, \sigma^2)$	TRIVIAL — it is just a Gaussian

Blur any distribution enough and it becomes a Gaussian. All intricate structure washes out. This is the entire basis of the field.

The plan

1. Sample from q_{large} . Easy — it is a Gaussian. `torch.randn()`.
2. Convert that into a sample of a slightly less blurred q .
3. Repeat down the ladder.
4. Arrive at a sample of p_{data} .

Each rung is easy *because the two distributions are nearly identical*. You never solve the hard problem — you solve a thousand almost-trivial ones.

Exactly one thing must be learned

Step 2 asks: given a noisy image, produce a slightly less noisy one. By Bayes:

$$q(\text{cleaner} \mid \text{noisier}) \propto q(\text{noisier} \mid \text{cleaner}) \cdot q(\text{cleaner})$$

└ you KNOW this ─┐ ┌ unknown ─┐
(it is your own noise)

For a small step that posterior is approximately Gaussian, and its mean is determined by a single quantity:

$E[x_0 | x_t] = \text{"given this noisy image, what was the clean image, on average?"}$

That is the only thing your network ever learns.

MSE training gives it to you for free

The minimiser of a squared loss against a random target is the conditional mean. Therefore:

$$\text{minimise } E \|D(x_t) - x_0\|^2 \quad \Rightarrow \quad D(x_t) = E[x_0 | x_t]$$

A plain MSE denoiser **is** the conditional expectation. No ELBO, no score function, no special objective required.

The five-line summary

1. Blur `p_data` by increasing amounts $\rightarrow$ a ladder from data to a Gaussian
2. The top of the ladder is trivially samplable
3. Descend by repeatedly asking "what was the clean image, on average?"
4. A plain MSE denoiser answers that question exactly
5. Re-inject a little noise each step so you `SAMPLE` rather than collapse onto the average

Step 5 matters more than it looks. Without it you walk straight to $E[x_0 | x_t]$ — the blurry mean of every compatible image. **The noise is what forces commitment to one specific image.** This is also precisely why aggressive 1-step distillation tends to blur: there is no room left to commit.

Where flow matching sits

Same ladder, different bookkeeping. Instead of “add noise of width σ ”, define a **coupling** over pairs and interpolate:

```
x0 = sample_from_data()
eps = torch.randn_like(x0)
t = sample_timestep()           # see §II

x_t = (1 - t) * x0 + t * eps     # this IS "adding the noise". One line.
target = eps - x0               # what you regress against
```

No `betas`, no `alphas_cumprod`, no lookup tables.

It is provably the same ladder

$(1-t)x_0 + t\epsilon$ with Gaussian ϵ is a scaled `p_data` convolved with a Gaussian of width `t`. The learned quantities are related by pure algebra. Since $\epsilon - x_0 = (x_t - x_0)/t$:

$$\underbrace{E[\epsilon - x_0 | x_t]}_{\text{flow matching}} = \underbrace{(x_t - E[x_0 | x_t])}_{\text{diffusion}} / t$$

Flow matching's velocity is "how far you are from your best guess of the clean image", rescaled by t . Identical information, different packaging.

Probabilistically there is **one idea**: blur the data into a Gaussian, learn $E[x_0 | x_t]$, walk back down. Diffusion and flow matching are two coordinate systems on that idea. Flow matching's coordinates simply have fewer places to make an off-by-one error.

The geometry: why paths curve

Diffusion paths are arcs — this is forced, not chosen

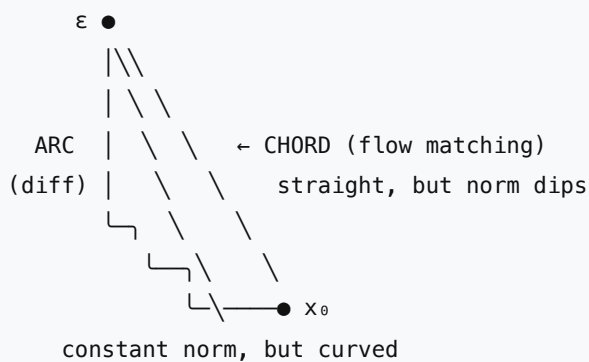
The general diffusion form is $x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{(1-\alpha_t)} \cdot \epsilon$. Square the coefficients and add:

$$(\sqrt{\alpha})^2 + (\sqrt{(1-\alpha)})^2 = \alpha + (1-\alpha) = 1 \quad \text{always, for any schedule}$$

That is what *variance preserving* means. And $a^2 + b^2 = 1$ **is the unit circle**, so any such pair can be written $(\cos \theta, \sin \theta)$. The trigonometry is not a design choice — it is what unit variance forces. The cosine schedule picks the simplest option, $\theta(t) = t \cdot \pi/2$:

$$x_t = \cos(t \cdot \pi/2) \cdot x_0 + \sin(t \cdot \pi/2) \cdot \epsilon$$

Flow matching's coefficients are $(1-t, t)$. At $t=0.5$ that gives $0.25 + 0.25 = 0.5 \neq 1$ — **not on the unit circle**. The norm sags in the middle; flow matching is *not* variance preserving.



You cannot have both. The arc is what constant norm looks like; the chord is what straight looks like. Diffusion chose constant variance. Flow matching chose straightness.

The proof from the derivatives

DIFFUSION, $\theta = t \cdot \pi/2$:

$$\begin{aligned} dx/dt &= (\pi/2) \cdot [-\sin(\theta) \cdot x_0 + \cos(\theta) \cdot \varepsilon] \\ &= (\pi/2) \times (x_t \text{ rotated by } 90^\circ) \end{aligned}$$

- velocity is PERPENDICULAR to position, magnitude proportional to it
- that is the definition of circular motion. The path MUST be an arc.

FLOW MATCHING:

$$\begin{aligned} dx/dt &= \varepsilon - x_0 && \leftarrow \text{no } t \text{ anywhere. CONSTANT.} \\ &\rightarrow \text{constant velocity} \Rightarrow \text{straight line, necessarily.} \end{aligned}$$

Why the arc costs nothing at training and everything at sampling

This is the single most important asymmetry in the document.

TRAINING: pick ONE random t . Compute x_t . One loss. Done.

You never visit two timesteps in a row.

You never travel anywhere.

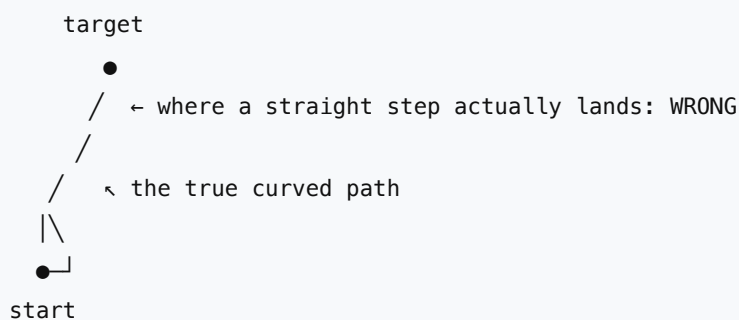
→ THE SHAPE OF THE PATH IS INVISIBLE TO YOU.

SAMPLING: start at $t=1$, must ARRIVE at $t=0$.

You must TRAVEL the whole path, in straight-line steps.

→ THE SHAPE OF THE PATH IS EVERYTHING.

Training samples **points**. Sampling traverses a **path**. Curvature is a property of paths, so it only bills you at sampling time.



On a straight path a straight step is exact — one step of any size, perfect. On a curved path every straight step drifts off, and you must take many small ones to keep up.

The two sources of curvature

They are curvature of two **different objects**. This distinction resolves most confusion in the field.

Source A — curvature of the *training path*

The line you interpolate along to build a training example. Purely geometric, no network involved, fully determined by your formula.

Diffusion: $x_t = \cos(\theta)x_0 + \sin(\theta)\varepsilon \rightarrow$ an ARC $\leftarrow$ curved
Flow matching: $x_t = (1-t)x_0 + t\varepsilon \rightarrow$ a straight LINE $\leftarrow$ not curved

Flow matching eliminates this completely and provably. It is a property of the definition.

Source B — curvature of the *learned field*

The path the ODE actually follows when generating. This depends on the learned field, and **both families have it**.

The model cannot see which (x_0, ε) pair produced the point it is looking at. So it predicts the **average** direction over all pairs passing through. Straight lines from random pairs **cross each other**, and at a crossing the model must emit *one* direction — so it emits the average, which matches neither.

Worked example (1-D, two data points)

Data: $x_0 \in \{-1, +1\}$ equally likely. $t = 0.5$, so $x_t = 0.5x_0 + 0.5\varepsilon$.

Every training line has constant velocity:

Pair A: $x_0 = +1, \varepsilon = -1 \rightarrow$ velocity = -2 (constant)
Pair B: $x_0 = -1, \varepsilon = +1 \rightarrow$ velocity = $+2$ (constant)

Both are perfectly straight. But both pass through $x = 0$ at $t = 0.5$, demanding two different velocities. The model outputs the likelihood-weighted average: **zero** — a direction no training example ever had.

Given x_0 , the distribution of x_t is $N(0.5x_0, 0.25)$, so by Bayes:

x_t	$P(x_0 = +1)$	$P(x_0 = -1)$	$E[x_0 x_t]$ — the target it aims at
0.0	50 %	50 %	0.00 — “somewhere between both”
0.3	77 %	23 %	0.54
0.6	92 %	8 %	0.83
0.9	97 %	3 %	0.95 — “definitely +1”

The average is taken over a set whose weights change as you move. So the average changes. So the velocity is not constant. So the path bends.

In plain words: at high noise the model does not know which mode it is heading for, so it aims at the average of all of them. As structure emerges it commits, and the aim swings toward the specific one. **You are steering toward a moving target, and steering toward a moving target is a curve by definition.**

Consequences:

- It is worst at **high noise** (maximum ambiguity, fastest swing).
- **Neither family escapes it** — it comes from averaging, not from the schedule.
- **Reflow** fixes it by making the coupling deterministic, so nothing crosses and there is nothing to average.

The summary table

Source	Diffusion	Flow matching	Fixed by
A — training path geometry (the arc)	present	eliminated	FM’s linear path, or DPM-Solver++’s exact integration
B — learned field averaging	present	present	only reflow or distillation

This is why distillation exists as an industry. No path choice and no solver removes Source B.

Terminal SNR: a bug you can see

At the final timestep, x_T should be **pure noise** — zero information about x_0 , because that is what you start from at inference (`torch.randn()`).

Diffusion gives $x_T = \sqrt{\alpha_T} \cdot x_0 + \sqrt{(1-\alpha_T)} \cdot \epsilon$. For pure noise you need $\alpha_T = 0$ exactly, but standard schedules end at $\alpha_T \approx 0.004$:

$\sqrt{0.004} \approx 0.06 \rightarrow 6\%$ of x_0 is still present in the "pure noise"

TRAINING: final timestep contains 6 % of the real image
INFERENCE: you start from `randn()` – contains 0 % of anything
↑ mismatch at the very first step you take

That 6 % carries the image’s overall brightness — the lowest-frequency component, which survives noise best. So the model learns to *read* brightness from the leak rather than *invent* it. At inference there is no leak, so it defaults to mid-grey.

Observable symptom: SD 1.5 cannot generate a truly black or truly white image. “A solid black background” produces dark grey; “pure white studio” produces light grey. Documented in Lin et al. 2023, *Common Diffusion Noise Schedules and Sample Steps are Flawed*. It reproduces.

Flow matching at $t = 1$: $(0) \cdot x_0 + 1 \cdot \epsilon = \epsilon$ **exactly**. Zero leakage by construction — the bug is not fixed, it is *unable to exist*.

But then what does the model learn at $t = 1$?

A common and reasonable confusion. Work out the target:

$x_1 = \epsilon$ pure noise, zero information about x_0
 $\text{target} = \epsilon - x_0$

The model must predict: $E[\epsilon - x_0 \mid x = \epsilon]$

Since ϵ tells you nothing about x_0 :

$$\begin{aligned}
 &= \epsilon - E[x_0] \\
 &= \epsilon - (\text{dataset mean image})
 \end{aligned}$$

The correct answer at $t = 1$ is “move away from noise, toward the average image”. Completely learnable, and it requires no knowledge of *which* image — because at that point there is none.

	Training at max noise	Inference at max noise	Consistent?
Diffusion ($\bar{\alpha}_T = 0.004$)	6 % of the real image present	0 % present	no
Flow matching	0 % present	0 % present	yes

Flow matching is *honest*. Diffusion is *dishonest* — the model optimises for a crutch that gets removed at test time. The process then bootstraps: step down from $t=1$, x_t gains structure, the prediction sharpens from “average image” → “average dog” → “this dog”.

Diffusion’s patch — zero-terminal-SNR (rescale so $\bar{\alpha}_T = 0$, plus v-prediction and trailing timestep spacing) — works fine. It is simply a patch you must know about and remember to apply.

SDE, ODE, and why deterministic sampling is valid

The three equations

FORWARD SDE (adding noise):

$$dx = f(x, t) dt + g(t) dW \quad \leftarrow dW = \text{randomness}$$

REVERSE SDE (generating, stochastic):

$$dx = [f - g^2 \cdot \nabla \log p] dt + g(t) d\tilde{W} \quad \leftarrow \text{still random}$$

PROBABILITY-FLOW ODE (generating, deterministic):

$$dx = [f - \tfrac{1}{2}g^2 \cdot \nabla \log p] dt \quad \leftarrow \text{NO } dW \text{ at all}$$

The network estimates $\nabla \log p$ (the score). **The ODE has no random term** — that is what makes it an ODE and what makes solvers applicable.

Proof that the ODE has the same marginals

The claim is about $p_t(x)$ only. Start from the **Fokker-Planck equation**, which describes how a density evolves under an SDE:

$$\begin{aligned}
 \partial p / \partial t &= \underbrace{-\nabla \cdot (f p)}_{\text{transport}} + \underbrace{\tfrac{1}{2} g^2 \Delta p}_{\text{diffusion}}
 \end{aligned}$$

The one algebraic trick. Use $\nabla \log p = \nabla p / p$, i.e. $\nabla p = p \nabla \log p$:

$$\begin{aligned} \frac{1}{2} g^2 \Delta p &= \frac{1}{2} g^2 \nabla \cdot (\nabla p) && \text{[definition of } \Delta] \\ &= \frac{1}{2} g^2 \nabla \cdot (p \nabla \log p) && \text{[substitute]} \\ &= \nabla \cdot \left(\frac{1}{2} g^2 p \nabla \log p \right) && \text{[} g^2 \text{ has no } x \text{ dependence]} \end{aligned}$$

The diffusion term is now a **divergence** — the same form as the transport term. Combine:

$$\begin{aligned} \partial p / \partial t &= -\nabla \cdot (f p) + \nabla \cdot \left(\frac{1}{2} g^2 p \nabla \log p \right) \\ &= -\nabla \cdot \left(f p - \frac{1}{2} g^2 p \nabla \log p \right) \\ &= -\nabla \cdot \left(\underbrace{f - \frac{1}{2} g^2 \nabla \log p}_v \cdot p \right) \\ &\quad \text{call this } v \end{aligned}$$

The **continuity equation** — how a density evolves under a purely deterministic flow $dx/dt = v(x, t)$ — is $\partial p / \partial t = -\nabla \cdot (v p)$. **That is exactly what we derived.**

Same initial condition + same evolution equation $\Rightarrow$ **p_t is identical for every t .** ■

What was and was not proven

PROVEN: p_t is identical for all t . Sample either, same distribution.
NOT TRUE: individual trajectories match.

Start the SDE and the ODE from the **same** x at $t = 1$ and they land in **different places**. Only the *distribution* over endpoints agrees.

The intuition

Re-read the trick: **diffusion of a density is indistinguishable from deterministic transport away from high-density regions**. The term $-\frac{1}{2} g^2 \nabla \log p$ points *down* the density gradient — particles pushed apart deliberately at exactly the rate random jitter would have spread them.

A crowd leaving a stadium. Version A: everyone wanders randomly; the crowd spreads. Version B: everyone is handed a specific direction, computed so the crowd spreads at *precisely the same rate*. Photograph the crowd at any moment — identical density. Follow any one person — completely different path.

Why this matters practically

Property	Consequence
It is a genuine ODE	Every numerical integrator in the textbook is available
Reproducible	Same seed → the same image, exactly
Invertible	Run it backwards → image-to-noise. Required for inversion-based editing
Exact likelihoods	ODEs admit change-of-variables; SDEs do not
Distillable	A deterministic map noise→image is a well-defined function you can regress against

A one-parameter family sits in between. Choose how much noise to re-inject: $\lambda = 0$ is the pure ODE, $\lambda = 1$ recovers the standard reverse SDE, $\lambda > 1$ adds extra churn (EDM’s `S_churn`). All share the same marginals. This is the ODE↔SDE dial, and it is a *sampling-time* choice — one trained model gives you the whole family.

Naming: flow matching, rectified flow, stochastic interpolants

They are the same construction. Three groups published it within weeks in 2022:

Name	Group
Flow Matching	Lipman et al. (Meta)
Rectified Flow	Liu et al. (UT Austin)
Stochastic Interpolants	Albergo & Vanden-Eijnden (NYU)

For a linear path with Gaussian noise, **all three produce an identical training objective**. The names reflect research communities, not different mathematics.

The *only* additional thing in the rectified-flow paper is **reflow** — the iterative straightening procedure. **SD3 and Flux do not use it**; they say “rectified flow” because they follow Liu’s naming. When you see the term in a model card, read it as “linear-path flow matching”.

Three names under one heading, and people conflate all three. *Rectified flow* is the **formulation** (linear path, velocity target) — alive and standard. *Logit-normal / shifted timestep sampling* is a **training detail** (which `t` you draw) — cheap and it matters. *Reflow* is an **iterative procedure** — superseded, skip it. Only the third one is dead.

What space does diffusion run in?

Everything above describes a process over *some* tensor. Which tensor is a separate decision, and it is the largest architectural fork in the field — it sets your token count, and therefore your attention cost, and therefore your latency floor.

Space	Encoder	Used by	Character
Pixels	none	Imagen, DIAMOND	Maximum detail fidelity, maximum compute per pixel
Continuous latent (VAE)	conv, trained <i>jointly with</i> the decoder for reconstruction	Stable Diffusion, SD3, Flux, GameNGen, Matrix-Game	Aggressive down/upsampling (typically 8× spatial, 4× temporal for video)
Discrete latent (VQ)	conv + codebook	IRIS-style tokenizers	Quantisation drops detail; an AR transformer predicts token indices
Representation latent (RAE)	frozen self-supervised ViT + trained ViT decoder	MIRA	Semantic, smooth, no spatial compression

A VAE is not a discrete tokenizer. Continuous latents lose far less detail than quantised tokens. When a paper argues “pixels beat latents on fine detail” (DIAMOND does), check whether it means *discrete* latents — usually it does, and the argument does not transfer to VAEs.

Representation autoencoders — the new result

The assumption for a decade was that a *reconstruction*-trained encoder is required, because semantic encoders throw away the detail you need to decode. [RAE \(ICLR 2026\)](#) — from Saining Xie’s group, who also produced DiT — showed otherwise. Freeze a **DINOv2/v3 or SigLIP2** encoder, train **only a ViT decoder** on top:

- Reconstructions **on par with, or better than, SD-VAE**
- **~6× fewer encoder GFLOPs and ~3× fewer decoder GFLOPs**
- Structurally: VAE = conv with aggressive compression; RAE = ViT with **no spatial compression** (the token grid comes from patching, not downsampling)

Why it works, and this is the part that generalises. A VAE encoder is optimised for reconstruction, so its latent is dense with high-frequency detail: **easy to decode, hard to predict**. A DINO encoder is optimised for semantics, so its space is smoother and more linearly structured — an easier target for a transformer to regress. The diffusion model’s residual errors get smaller and less structured, and **small structured errors are exactly what compound into drift** (Part VI).

This is the same insight as **REPA** (add an auxiliary loss aligning DiT internals to DINO features, converge much faster), pushed to its conclusion: *don’t align to representations — diffuse in them*.

Vocabulary: *DINOv3* is a specific pretrained model. *RAE* is the architecture pattern. MIRA uses **an RAE whose encoder is DINOv3-L/16**. Both statements are true, at different levels.

Two practical traps

⚠ **The scaling factor.** A VAE latent must be multiplied by `vae.config.scaling_factor` before diffusion and divided after. Skip it and your model trains against the wrong variance — it will still converge to *something*, which is why this bug survives so long. A representation encoder needs *its own* normalisation statistics, not SD’s.

⚠ **Your images may be too small for a latent experiment at all.** An 8× VAE turns 32×32 (CIFAR) into a 4×4 latent. Patchify that at `p=2` and your transformer sees **4 tokens**. Any conclusion you draw is noise. You need $\geq 64 \times 64$ inputs so the latent is $\geq 8 \times 8$. This costs nothing — diffusion on an 8×8×4 latent is *cheaper* per step

than on 32×32 pixels.

The DiT backbone

“DiT is the architecture, flow matching is the objective” is the standard summary, and every model in this handbook’s landscape table is a DiT. Here is what that actually means, because **replacing a UNet with a transformer is not a module swap** — it is six changes with no UNet analogue.

1. Patchification is now your primary compute knob. The image/latent is cut into $p \times p$ patches, each linearly projected to a token. DiT’s headline result: **shrinking p improves FID at fixed parameter count**. That is pure FLOPs, not capacity — a fact with no UNet equivalent, and it means you sweep p before you sweep width or depth.

2. Conditioning becomes a design decision. A UNet gives you FiLM-style normalisation modulation and that’s mostly that. A DiT offers four options:

Mechanism	How	Used by
adaLN-Zero	timestep/class vector $\rightarrow$ per-block scale & shift on LayerNorm, zero-initialised	Original DiT — won its ablation
Cross-attention	condition as keys/values in a separate attention op	SD3-era UNets, many video models
In-context tokens	prepend the condition as extra tokens	Simple, weaker in DiT’s tests
MMDiT (joint two-stream)	text and image tokens in one attention op with separate per-modality weights	SD3, Flux

Conditioning in a DiT does not necessarily go through attention. adaLN modulates normalisation. **Attention is for tokens; adaLN is for scalars and vectors** (timestep, class, guidance strength). This trips people constantly.

3. You lose the multi-scale hierarchy and the skip connections. The UNet’s locality bias and multi-resolution structure were free inductive bias. A DiT has neither $\rightarrow$ slower convergence and more data hunger. U-ViT’s long skip connections exist precisely to give some of it back.

4. Positional encoding decides resolution generalisation. Absolute sincos locks you to trained shapes; **2D/3D RoPE** lets you extrapolate. (Part II covers the mechanics and the interpolation/NTK fixes.)

5. Attention is $O(N^2)$ and $N \propto (\text{res}/p)^2$. This is why a pixel-space DiT is quadratically punished, and therefore why **DiT presupposes a latent space**. It is also why a small pixel-space world model (DIAMOND runs at 56×30) has no reason to switch.

6. It scales predictably. DiT’s real contribution was a clean FLOPs $\rightarrow$ FID curve. If you build one, reproduce that plot — it is simultaneously an architecture lesson and a GPU-utilisation lesson.

Attention itself is ordinary — the mask and the token set are the whole game

The operator is standard scaled-dot-product attention everywhere (FlashAttention in practice). Nobody invents exotic attention for diffusion. What varies:

Setting	Mask	Tokens attended
Image DiT	bidirectional (none)	all patches of one image
Text-to-image	bidirectional	patches + text (cross-attn, or MMDiT joint)
Video DiT, cheap	bidirectional, factorised	spatial within frame, then temporal across frames
Video DiT, modern	bidirectional, full 3D	all space-time tokens at once — better, costlier
Interactive / streaming	block-causal + KV cache	causal across chunks, bidirectional within a chunk
+ retrieval memory	block-causal, joint	memory + past + noisy-current latents in one attention space

Seeing it this way collapses a dozen “architectures” into one operator with two configuration choices.

The zero-init pattern, which recurs everywhere. `adaLN-Zero`, ControlNet’s zero-initialised joins (Part II), and DIAMOND’s `nn.init.zeros_(self.conv_out.weight)` are the same idea: **start the new component as an exact no-op, so adding capacity cannot make things worse**. Whenever you bolt something onto a working model, zero-init the join.

Part II — Training recipe (flow matching)

Why flow matching, honestly ranked

Rank	Reason	Weight
1	Simpler parameterisation — no β schedule, no $\bar{\alpha}$ tables, no $\epsilon/x_0/v$ decision. Far fewer bugs.	biggest
2	Correct terminal condition — no terminal-SNR bug, structurally	big
3	Where the recipes and tooling live — SD3, Flux, Wan, HunyuanVideo, Mochi, LTX	big
4	Lower curvature (Source A eliminated)	modest

Straightness is the theoretical selling point. **Bug-resistance is the practical one.**

The complete training loop

```
# ----- 1. TIMESTEP: logit-normal, NOT uniform -----
u = torch.randn(B, device=dev) * s + m          # SD3 default: m=0.0, s=1.0
t = torch.sigmoid(u)                             # bell curve on (0,1)

# ----- 2. RESOLUTION SHIFT (critical if multi-resolution) -----
shift = compute_shift(seq_len)                   # see §II.4
t = shift * t / (1 + (shift - 1) * t)

# ----- 3. LINEAR PATH -----
t_ = t.view(-1, 1, 1, 1)
noise = torch.randn_like(x0)
x_t = (1 - t_) * x0 + t_ * noise

# ----- 4. VELOCITY TARGET (constant along the path) -----
target = noise - x0

# ----- 5. CONDITION DROPOUT (enables CFG later; costs nothing) -----
mask = torch.rand(B, device=dev) < 0.10
cond = torch.where(mask[:, None], null_embedding, cond)

# ----- 6. PLAIN MSE. No loss weighting needed. -----
loss = F.mse_loss(model(x_t, t, cond), target)
```

Plus, non-negotiably: **bf16** and **EMA weights**.

Timestep distributions and why they are bell-shaped

What logit-normal is

Sample a normal, squash it into (0,1) with a sigmoid:

```
u = torch.randn(B) * s + m
t = torch.sigmoid(u)
```

Called *logit*-normal because $\text{logit}(t) = \log(t/(1-t))$ — the inverse of sigmoid — is what is normally distributed.

EDM uses lognormal instead — purely a domain difference

EDM parameterises by $\sigma \in (0, \infty)$, unbounded above, so it needs a distribution on the positive reals $\rightarrow$ lognormal. Flow matching uses $t \in [0, 1]$, bounded $\rightarrow$ logit-normal. Both are “Gaussian after a change of coordinates”.

They are the same choice

The natural coordinate for both is **log-SNR**. For the rectified-flow path:

```
 $\lambda = 2 \cdot \log((1-t)/t) = -2 \cdot \text{logit}(t)$ 
```

So **logit-normal in t is exactly Gaussian in log-SNR**. EDM’s lognormal in σ is also Gaussian in log-SNR. Two groups, two coordinate systems, one underlying decision.

Why a bell curve is correct

log-SNR	Task	Learning value
Very high ($t \rightarrow 0$)	remove faint grain	~nothing
Middle	dog or cat? where is the horizon? what is the composition?	everything
Very low ($t \rightarrow 1$)	output the dataset mean	~nothing

Uniform sampling burns most of your gradient steps on the two useless ends. s controls how aggressively you concentrate.

Model	Distribution
EDM	$\log \sigma \sim N(-1.2, 1.2^2)$
SD3 / Flux	logit-normal, $m=0, s=1$
Never	uniform

Resolution shift — the single most common serious bug

The mechanism, and it is counter-intuitive

At high resolution the training task becomes **too easy**, so the model never learns the hard part.

A given real-world feature spans more pixels at higher resolution — an eyelash covering 1 pixel at 256 px covers 16 pixels at 1024 px. So neighbouring pixels are far more correlated. **Noise is added i.i.d. per pixel; signal is correlated across pixels.** Averaging a neighbourhood therefore cancels noise while preserving signal:

Average a 4x4 block: noise σ drops by $\sqrt{16} = 4\times$
signal survives (nearly constant across the block)

Result: the same σ is effectively $\sim 4\times$ weaker at $4\times$ the resolution.

So during training the model can always still see the composition at any given σ . **The gradient signal for “decide the layout from nothing” never fires.** Then at inference you start from pure noise, where there genuinely is nothing to read — the one thing it never practised.

Symptom: gorgeous local texture, broken global structure. Two heads. Duplicated torsos. A subject repeated across the canvas with no coherent scene. This is exactly the classic SD 1.5-at-1024px failure.

The formula

$$t_{\text{shifted}} = \text{shift} * t / (1 + (\text{shift} - 1) * t)$$

A monotone remap of $[0,1]$ onto itself that pins both endpoints and pushes the middle upward. With `shift = 3`:

you sample	becomes	
0.00	0.00	endpoint pinned
0.25	0.50	
0.50	0.75	median moves well up
0.75	0.90	
1.00	1.00	endpoint pinned

Why *this* formula specifically

It multiplies the odds by exactly `shift`:

$$t'/(1-t') = \text{shift} \cdot t/(1-t)$$

Since $\lambda = 2 \cdot \log((1-t)/t)$, that is:

$$\lambda' = \lambda - 2 \cdot \log(\text{shift})$$

A pure constant translation of the log-SNR distribution. That is the correct knob, because the resolution effect is approximately a constant log-SNR offset. Elegant rather than arbitrary.

How Flux computes it in practice

```
# linear in sequence length, then exponentiate
m = (max_shift - base_shift) / (max_seq_len - base_seq_len)
b = base_shift - m * base_seq_len
mu = image_seq_len * m + b
shift = math.exp(mu)

# applied to every sigma in the schedule
sigma_shifted = shift * sigma / (shift * sigma + 1 - sigma)
```

With Flux’s config (base_seq_len=256, max_seq_len=4096, base_shift=0.5, max_shift=1.15):

tokens	≈ resolution	mu	shift
256	256 px	0.50	1.65
1024	512 px	0.67	1.96
4096	1024 px	1.15	3.16

Applied at both training and inference — for different reasons

	Purpose	Consequence if wrong
Training-time shift	decides which noise levels get trained	model never learns to invent structure → broken composition
Inference-time shift	decides which sigmas your step budget visits	you spend steps where nothing happens

Match them. Flux recomputes shift from the requested image size on every call, so changing output resolution changes the schedule automatically.

Debugging rule: if 1024 px generations show duplicated subjects or incoherent layout, check shift before anything else.

Flow matching needs this too

shift is literally SD3’s and Flux’s parameter, and both are flow matching models. It was not invented to patch diffusion. This is a property of images at different resolutions, not of any generative formulation — diffusion, flow matching, and EDM are equally affected.

Classifier-free guidance and condition dropout

The problem

Train on (image, caption) pairs and sample: the output only *loosely* follows the caption. Thousands of images are “compatible” with “a red car on a beach”, including ones where the car is barely red. The model samples from all of them evenly.

Step 1 — condition dropout, at training time

```
if random() < 0.10:
    caption = ""           # ← the entire "10 % condition dropout"
```

One model now knows two jobs: *generate matching this caption* and *generate anything*.

This is what makes it “classifier-free”. Earlier methods trained a *separate* classifier or unconditional model. Dropout collapses two models into one, at the cost of blanking a text embedding 10 % of the time. Without it, `model(x, t, "")` is undefined and CFG is impossible.

Step 2 — extrapolate, at sampling time

```
v_cond    = model(x, t, caption)
v_uncond  = model(x, t, "")

v = v_uncond + w * (v_cond - v_uncond)
```

```
v_uncond    → "toward any plausible image"
v_cond      → "toward any plausible image matching the caption"
difference   → "the caption-specific component, ISOLATED"
```

Subtraction removes everything both have in common (be an image, be sharp, be coherent) and leaves only the caption’s contribution. Then you amplify *that*.

Why not just use `v_cond`?

You can — that is `w = 1` exactly. The point is going *past* it:

w	Result
0	ignores the caption entirely
1	plain conditional — weak adherence
3.5–5	sweet spot for flow matching (Flux $\approx$ 3.5)
7–8	sweet spot for SD 1.5 / SDXL
12+	fried — oversaturated, neon, low diversity

High `w` breaks because you are extrapolating far past where the score estimate is valid. Amplify a direction 15× and you leave the data manifold.

The classifier hiding inside

By Bayes, $\log p(c|x) = \log p(x|c) - \log p(x) + \text{const}$. Take the gradient:

$$\nabla \log p(\text{caption} | x) = v_{\text{cond}} - v_{\text{uncond}}$$

That difference is the gradient of a classifier you never trained. Amplifying it by w steers toward images the implicit classifier is confident about.

The cost, and why it matters downstream

CFG doubles your compute — two forward passes per step. “25 steps” really means **50 network evaluations**. This is why guidance distillation (§IV) exists as a separate $2\times$ win.

Practical weight: without CFG, text-to-image output is not merely worse, it is largely unusable. This is load-bearing, not a fine-tuning knob.

What the guidance scale costs you — measure it, don’t eyeball it

w trades **fidelity up, diversity down**. Sweep it and plot **FID and Recall on the same axes**: FID improves then flattens, while Recall falls monotonically. This is the cleanest available demonstration that a single scalar metric lies to you (Part VIII), and it takes an afternoon.

Guidance intervals — apply guidance only in a middle band of t , $w=1$ at the extremes — frequently beat a constant w , at zero cost.

The same dropout trick, applied to other conditions

Condition dropout is not text-specific. It is a general recipe: **drop a condition during training and you get an unconditional branch plus a guidance dial for that condition, free.**

The clearest non-text example is **action dropout** in interactive world models (Part VII). MIRA randomly hides player actions during training and gets two capabilities it never had to design:

1. The model learns $p(\text{next frame} | \emptyset)$ alongside $p(\text{next frame} | \text{action})$. **The unconditional branch is autopilot** — the world plays itself with no controller attached.
2. **Action-CFG:** $v_{\emptyset} + w \cdot (v_{\text{action}} - v_{\emptyset})$. Raise w and actions become *more* influential than they were in the training data — a controllability knob that falls straight out of the dropout.

If you condition on anything — camera pose, depth, actions, audio — drop it during training. Nearly free, and you will want the dial later.

Conditioning a frozen model: ControlNet and the adapter family

CFG steers a model using conditions it was **trained with**. Different problem: you have a good pretrained model and want to add a **new** conditioning signal — edges, depth, pose, camera, actions — without retraining it.

⚠ **These two get conflated constantly, and they are not the same mechanism:**

	Adapter family (ControlNet et al.)	Guidance (CFG, classifier guidance)
Adds parameters?	Yes — a new branch you train	No
Needs training data?	Yes — (condition, output) pairs	No
Mechanism	New network injects features into the frozen base	Reweights predictions at sample time
When	Once, offline	Every sampling step
Inference cost	One slightly larger forward	Two forwards

Three canonical forms of the trained kind:

Method	Mechanism	Cost
ControlNet	Clone the base encoder blocks, train the copy on the new condition, inject through zero-initialised connections, freeze the base	Heaviest — roughly half a model
T2I-Adapter	A small dedicated side network whose features are added into the base	Much lighter, slightly less faithful
IP-Adapter	Decoupled cross-attention for <i>image</i> prompts	Light; specific to image conditioning

Why build a ControlNet even if you never ship one. The zero-init clone is precisely the pattern used to add **action conditioning to a pretrained video DiT** — Matrix-Game 2.0 takes a pretrained Wan-family DiT and injects **action modules into every DiT block**. Build a ControlNet on an image model and you have already built the mechanism; only the conditioning signal changes, from an edge map to a controller state. It is the cheap rehearsal for Part VII.

Practical note: train it on *your own small model*, not on SDXL. Same lesson — freeze the base, add a branch, zero-init the join — at $\sim 1\%$ of the compute.

LoRA / PEFT — adjacent tooling, unrelated mechanism

LoRA has nothing to do with guidance. It is *parameter-efficient fine-tuning*: freeze the base, learn a low-rank update $\Delta W = BA$ (usually on attention projections), merge or hot-swap at inference. It shares a conversation with ControlNet because both adapt a frozen model, but the mechanisms are unrelated.

Three reasons it matters: it is how fine-tunes are distributed; **LCM-LoRA** made *step distillation itself* a swappable adapter, which is genuinely surprising (Part IV); and it is how you adapt a large pretrained model on a small budget.

Multi-resolution training

Both components are already resolution-agnostic

The VAE is **convolutional**, so it does not care about input size:

```
256×256 → VAE → 32×32 latents
512×512 → VAE → 64×64 latents
1024×1024 → VAE → 128×128 latents
```

One VAE, same weights, all resolutions. No per-resolution variants.

The DiT is a transformer over patches, so resolution just means a different sequence length:

```
32×32 latents, patch 2 → 256 tokens
64×64 latents, patch 2 → 1,024 tokens
128×128 latents, patch 2 → 4,096 tokens
```

Transformers handle variable sequence length natively. **This is a large part of why the field moved from UNets to DiTs.**

The one thing that must be right: positional encoding

```
Learned absolute embeddings  - a fixed TABLE. Position 5000 has no row.
                              BREAKS at new resolutions.

2D RoPE (rotary)             - a FUNCTION of position. Extends and
                              interpolates gracefully.
```

Every modern DiT uses RoPE. If you are building one, this single choice determines whether multi-resolution is possible at all.

How RoPE encodes relative position

Rotate the query at position m and the key at position n . Because $R(a)R(b) = R(a+b)$ and $R(\theta)^T = R(-\theta)$:

```
score = [R(m)q]^T [R(n)k] = q^T R(m)^T R(n) k = q^T R(n-m) k
                                     ↑
                               only the DIFFERENCE survives
```

Absolute positions cancel algebraically. For images, split the head dimension in half — rotate one half by row index, the other by column index — so scores depend on $(\Delta_{row}, \Delta_{col})$. “Three patches left and two up” means the same thing at 512 px and 1024 px.

Length / resolution extrapolation

RoPE does not hard-fail like a table, but it **degrades**:

```
train 2048 → test 2048   fine
train 2048 → test 4096   noticeably worse
train 2048 → test 8192   usually unusable untreated
```

Two causes: **unseen rotation angles** (low-frequency dimensions barely rotated during training and swing into uninterpreted territory), and **attention entropy growth** (more tokens to spread softmax mass over than the model was calibrated for).

Fix	Mechanism
Position Interpolation (PI)	divide positions by a scale factor — compress into the trained range. Blurs local precision.
NTK-aware scaling	scale frequencies <i>unequally</i> — high-frequency (local) barely, low-frequency (global) a lot. Preserves local precision.
YaRN	NTK-by-parts + attention temperature correction. Best current method; needs ~0.1 % of original tokens to fine-tune.
Fine-tune at the target length	Simplest and most reliable. This is what production does.
Train variable-length from the start	Best outcome, costs training efficiency.

RoPE turns “impossible” into “needs a short fine-tune”. It does not make extrapolation free.

The practical training procedure

- 1. **Aspect-ratio bucketing.** Tensors in a batch must share a shape, so group images into buckets by resolution/aspect and draw each batch from one bucket. Standard in SDXL, SD3, Flux, Wan. Also fixes “everything is square” outputs.
- 2. **Progressive resolution.** Semantics are resolution-independent — learn them where tokens are cheap:

Phase 1:	256 px	— ~80 % of steps.	Learns semantics, composition, concepts.
Phase 2:	512 px	— ~15 %.	
Phase 3:	1024 px	— ~5 %.	Learns detail and high-res structure.

- 3. **Size conditioning** (SDXL’s contribution). Feed original image dimensions and crop coordinates as extra conditioning. Without it, training on crops teaches the model to *produce* crops — heads cut off at the frame edge.
- 4. **Per-sample shift.** Compute `shift` from each sample’s own token count, so a mixed-bucket batch gets per-sample shifts.

Training checklist, ranked by impact

Priority	Decision
1	Logit-normal timesteps (never uniform)
2	Resolution shift — skip it and high-res output is structurally broken
3	EMA weights
4	10 % condition dropout (free; enables CFG)
5	2D RoPE, never a learned position table
6	Aspect-ratio bucketing + progressive resolution
7	bf16, plain MSE loss

Part III — Sampling and solvers

Do people use solvers? Always

Yes, 100 % of the time. There is no alternative.

The model never outputs an image. It outputs a **direction** — “from this noisy point, move this way.” Getting to an image means repeatedly asking for a direction and moving. The rule you use for “and moving” is the solver. Even the simplest possible loop ($x = x + v*dt$) is a solver — it is called Euler.

So the question is never *whether*, only *which*.

The error formula: everything follows from this

Forget diffusion; think of a car.

Symbol	Meaning	In sampling
h	how long you drive before re-checking	step size ($1/\text{num_steps}$)
v	current speed & direction	what the network returns
a	how fast the direction is <i>changing</i>	curvature

Taylor’s theorem gives the exact truth:

$$\text{position}(t+h) = \text{position} + \underbrace{h \cdot v}_{\text{Euler uses only this}} + \underbrace{\frac{1}{2}h^2 \cdot a}_{\text{Euler THROWS AWAY}} + (h^3/6) \cdot (\dots) + \dots$$

← this is the curvature term

Euler’s error is exactly the first term it discarded: $\frac{1}{2}h^2 \cdot a$. Not a heuristic — literally the discarded term.

Concrete numbers

Car at 100 km/h, turning, $a = 20$:

	h	h²	error = $\frac{1}{2} \cdot h^2 \cdot 20$
check every 1.0 s	1.0	1.00	10.0 m off
check every 0.5 s	0.5	0.25	2.5 m off
check every 0.1 s	0.1	0.01	0.1 m off

Halving **h** **quarters** the per-step error, because **h** is squared. You take twice as many steps, so total error **halves**. That is what “first order” means.

Second-order solvers: per-step error $\propto h^3$, total $\propto h^2$. Halve the steps and total error **quarters** instead of halving.

It is a product — kill either factor

$$\text{error} = \frac{1}{2} \cdot h^2 \cdot a$$

↑

take tiny steps

↑

OR drive on a straight road

If the road is straight, $a = 0$, and the error is zero for ANY h . Euler is *exact* on a straight line — one step, any size, perfect.

This is why “make the path straight” (flow matching) and “use a better solver” (DPM-Solver++) are two routes to the same destination.

Why Euler is the right choice for flow matching

This resolves an apparent contradiction: Euler is the *worst* solver, and it is also the *recommended* one for flow matching.

A solver’s only job is to correct for curvature. If there is little curvature and small steps, there is nothing to correct.

Regime	Curvature	Step size	Fancy solver?
Flow matching, 25 steps	low	small	No — use Euler
Flow matching, 8 steps	low	large	Yes
Diffusion, 25 steps	high	small	Yes, noticeably
Diffusion, 8 steps	high	large	Yes, a lot

Flow matching at 25 steps shrinks *both* factors: Source A curvature is eliminated at training time, and 25 steps keeps h small. **Multiply two small numbers and a 2nd-order solver would halve an error that is already invisible.**

The specific reason DPM-Solver++ does not transfer

DPM-Solver++’s central trick is not “be 2nd order”. It is integrating the known analytic part exactly. The diffusion ODE is semi-linear:

$$\frac{dx}{dt} = \underbrace{f(t) \cdot x}_{\text{KNOWN in closed form causes the arc}} + \underbrace{-\frac{1}{2}g(t)^2 \cdot \nabla \log p}_{\text{the network the data's pull}}$$

Naive Euler approximates **both**. DPM-Solver changes variables to log-SNR, where the linear term integrates to a clean `exp(...)` factor:

apply $\exp(\dots)$ ← EXACTLY, on paper. Zero error.
approximate the score ← the only thing left to estimate

An entire error category deleted for free. **That is the 5×**.

$f(t) \cdot x$ is the arc, in algebraic language. $dx/dt = c$ gives a straight line; $dx/dt = f(t) \cdot x$ means the velocity depends on position, so it changes as you move — curvature. Flow matching's velocity is $\varepsilon - x_0$, with no x on the right-hand side at all.

Diffusion: $dx/dt = [\text{BIG } f(t) \cdot x] + [\text{score}]$
↑ DPM++ zeroes this error → big win

Flow matching: $dx/dt = [\sim\text{nothing}] + [\text{velocity}]$
↑ zeroes an error already $\sim\text{zero}$ → no win

DPM-Solver++ on a flow matching model arrives to harvest a field flow matching already harvested.
Same win, collected two different ways — which is exactly why they do not stack.

Flow matching does NOT have zero curvature

An important correction to a natural inference. **Source B is fully intact.** Euler is *not* exact.

Proof: if flow matching paths were actually straight, one Euler step would be exact and nobody would ever need distillation. The entire few-step research field (DMD2, LADD, LCM, reflow) exists precisely because the trajectories are still curved.

Which is why Euler breaks below ~ 15 steps:

Steps	h	h^2	$h^2 \cdot a$	Verdict
30	0.033	0.0011	invisible	Euler fine
15	0.067	0.0044	borderline	Euler ok
8	0.125	0.0156	visible	need 2nd order
1	1.000	1.0	hopeless	distil

From 30 → 8 steps, h^2 grows **14×**. The residual curvature never changed — you simply stopped hiding it behind tiny steps.

Solver selection table

Your model	Use	Steps
Flow matching (Flux, SD3, Wan, Lucy)	Euler (FlowMatchEulerDiscrete)	20–30
Flow matching, under ~15 steps	flow-adapted DPM / Heun / res-multistep	8–15
Diffusion (SD 1.5, SDXL, EDM)	DPM-Solver++ 2M, Karras schedule	20–30
Diffusion alternative	UniPC	20–30
EDM-faithful	Heun (2nd order)	18–36
Distilled (LCM, Turbo, DMD2)	none — the method’s own scheduler	1–4

DPM-Solver++ 2M Karras is very likely the most-used sampler on Earth by volume.

Two things that matter more than solver choice

- 1. **The step schedule.** Karras / log-SNR spacing over uniform spacing is a bigger quality jump than any solver swap.
- 2. **Step count.** Going 15 → 25 steps helps more than swapping solvers at either count.

Solver choice is genuinely the least important item on the sampling checklist.

Step schedules

Steps are **not** decrements of 1. Two coordinate systems:

Legacy discrete models (T = 1000 integer timesteps) **stride**:

100 steps	→	1000, 990, 980, ...	decrements of 10
20 steps	→	1000, 950, 900, ...	decrements of 50

So “DPM-Solver++ at 20 steps ≈ DDIM at 100 steps” means: **jumps of 50 land as accurately as jumps of 10.**

Continuous models partition [0,1] (or $\sigma_{\max} \rightarrow \sigma_{\min}$), and the pieces are **deliberately unequal**:

uniform:	1.00	0.95	0.90	0.85	...	0.05	0.00	
Karras (p=7):	1.00	0.87	0.72	0.57	...	0.008	0.00	← crowded near the end

Early steps decide coarse structure and tolerate big jumps; late steps refine detail and need small ones.

SDE vs ODE at sampling time

	ODE (deterministic) — the default	SDE (stochastic)
Steps needed	fewer	more
Reproducible	yes	no
Invertible	yes (needed for editing/inversion)	no
Distillation teacher	required	unusable for trajectory methods
Error behaviour	integrates bias with no correction	self-corrects score error
Best at	low–moderate NFE	high NFE (100+), max fidelity
Also helps	—	mitigates high-CFG oversaturation

Reach for the SDE only when you have 100+ NFE and are chasing FID, or when high CFG is burning your outputs. In UIs, names containing `a`, `ancestral`, or `SDE` are stochastic.

Part IV — Distillation recipe

When to distil

20–50 steps needed → DO NOT DISTIL. Base model + good solver.
8–15 steps needed → Try base model + tuned solver + shift FIRST.
4–8 steps needed → Distil.
1–2 steps needed → Distil, and accept diversity loss.

Distillation is a last resort that costs you something real, not a default. And it is *downstream* of good solver work, never a substitute — **the base model plus its ODE solver is the teacher you distil from.**

Many teams reach for distillation and then discover a well-tuned 12-step Euler run with the right shift was already acceptable, at zero training cost.

Why determinism is required

Because the ODE has no randomness, this is a **function**:

$F : z \mapsto x$ "start at noise z , run 50 solver steps, get image x "

One input, exactly one output. Now consider training a student with a regression loss against a **stochastic** teacher:

Student minimises $\mathbb{E} \| \text{student}(z) - Y \|^2$ where Y is random

The minimiser of a squared loss against a random target is:

$\text{student}(z) = \mathbb{E}[Y \mid z]$ ← the CONDITIONAL MEAN

The average of several valid images is mud. Your student converges to blur — not from bad training, but because the blur *is* the correct minimiser of the loss you wrote.

Which methods actually require it

Method	Needs determinism?	Why
Progressive distillation	strictly	Target is literally “what the teacher’s 2 steps produced”
Consistency / LCM	strictly	Target is the trajectory endpoint
Reflow	strictly	The point is building a deterministic coupling
DMD (original)	yes	Included a regression term on teacher-generated pairs
DMD2	no	Dropped that term; uses score oracle + GAN loss

This is not incidental — it is why DMD2 can exceed its teacher while LCM cannot. Regressing against $F(z)$ caps you at F . Matching a distribution does not.

The three families

A. Trajectory distillation — “copy the teacher’s path, skip ahead”

Method	Mechanism
Progressive Distillation (Salimans & Ho 2022)	Student does in 1 step what the teacher does in 2. Repeat: 1000→500→250→...→4. Student remains a denoiser.
Consistency Models (Song et al. 2023)	Student maps <i>any</i> point on a trajectory directly to its endpoint. Enforced by a self-consistency loss between adjacent points.
LCM (Luo et al. 2023)	Consistency models in latent space. LCM-LoRA is the popular form.

Ceiling: the teacher. Achieved: below the teacher. Tends to blur at 1–2 steps because predicting a long jump’s endpoint regresses toward the mean of plausible endpoints.

Why LCM-LoRA works at all

LoRA freezes W and learns a low-rank correction $W + BA$ with $r \approx 4\text{--}64$ — about 1 % of parameters, originally for style fine-tuning.

The LCM-LoRA insight: “be a 4-step model instead of a 50-step model” is a **low-rank, content-agnostic** change. It is not about *what* the model draws, only *how far it jumps*. So the adapter **transfers across fine-tunes of the same base** — train once on SD 1.5, apply to any SD 1.5 checkpoint.

```
pipe.load_lora_weights("latent-consistency/lcm-lora-sdv1-5")
pipe.scheduler = LCMScheduler.from_config(pipe.scheduler.config)
image = pipe(prompt, num_inference_steps=4, guidance_scale=1.0).images[0]
```

~32 A100-hours instead of a full distillation run. The *objective* is still full consistency distillation; LoRA is only the parameterisation of the update. Quality sits somewhat below distilling each checkpoint directly.

B. Distribution matching — “forget the path, match the distribution”

DMD (Yin et al. 2023) does not imitate the trajectory at all. It makes the student’s **output distribution** match the data distribution, using two score networks:

```
real score = frozen teacher                → "where real data is"
fake score = a second net, continuously
              trained on the student's outputs → "where the student is"

gradient to student ∝ (fake score - real score)
```

That difference is a vector pointing *from* the student’s current distribution *toward* the data distribution — an approximate KL gradient. (Same machinery as VSD / ProlificDreamer from the text-to-3D literature.)

DMD2 (2024) is the practical version: drops the expensive pre-generated pair dataset, adds a GAN loss, supports 4 steps, and uses a two-timescale update to stabilise the fake-score network.

Ceiling: none from the teacher. The teacher’s own *samples* are worse than its *score field* — a 50-step run accumulates 50 steps of discretisation and score-approximation error. The student never runs that trajectory, so it can land closer to the data than any actual teacher sample.

A mediocre pianist can still have perfect ears. Copy their *performance* and you cap out at mediocre. Learn from their *judgement* and you can surpass them.

Caveat: “exceeds the teacher” is measured on **distributional metrics like FID**, and the student directly optimises a distributional objective while the teacher optimises per-sample denoising. On **diversity / recall**, few-step students still typically lose.

C. Adversarial distillation

Method	Mechanism
ADD (SDXL-Turbo)	Score distillation + GAN discriminator on pixels (DINOv2 features)
LADD (SD3-Turbo)	Discriminator moved into latent space , using the teacher’s own features as backbone. Cheaper, higher resolution, multi-aspect.

Very sharp at 1–4 steps. Classic GAN trade-off: **you lose output diversity**.

In practice B and C have merged — DMD2 uses a GAN loss, LADD uses distillation losses. Modern recipes are hybrids.

Method selection

Steps needed	Method	Cost	Trade-off
8–15	none — tune the solver	zero	none
4–8	DMD2	heavy	best quality; can beat teacher
4–8	LCM-LoRA	~32 A100-hr	cheapest; blurrier
1–4	DMD2 or LADD	heavy	sharp; diversity loss

Why a GAN loss is safe here

A reasonable objection: GANs are notoriously unstable. Four things differ:

1. **The student is not starting from zero.** It is initialised from the teacher and already produces good images on day one. The discriminator never gets the trivial early win that kills classic GAN training. **The GAN is a finishing pass, not the thing doing the learning.**
2. **There is always an anchor loss.** DMD2’s distribution-matching gradient; LADD’s score distillation. The student is tethered to the teacher’s distribution and cannot mode-collapse away.
3. **The discriminator is pretrained** — DINOv2 or the teacher’s own DiT features. Strong representations give useful gradients immediately.
4. **It is doing a narrow job** — not “learn the data distribution”, just “stop being blurry”.

Why you need it

Distillation losses are **regression** losses. When several outputs are plausible, an L2 loss returns their **average**, and the average of plausible images is blurry. That is precisely the failure mode of few-step distillation.

A discriminator’s entire job is to say “*this looks like an average, not a sample.*” It is the exact right tool for the exact defect.

What you still pay: diversity. The student learns to emit the “safest” sharp output. Measured as lower recall / mode coverage. This is the standing trade-off of every Turbo-class model.

The full family map — and which ones you actually need

The three families above are the ones you implement. The literature has roughly seven, and knowing which is which stops you chasing papers you will never use.

Family	Members	Verdict
Distribution matching	DMD → DMD2	Implement. The workhorse. DMD2 drops the regression term and adds a GAN objective so it can learn from real data. Every real-time interactive model uses a DMD-family loss
Causal / rollout	CausVid → Self-Forcing → Causal Forcing / ++	Implement if you stream. Simultaneously a rollout scheme and a distillation objective (Part VI)
Guidance distillation	(below)	Implement. Nearly free 2×, and universally forgotten
Consistency	CM → LCM / LCM-LoRA → iCT/ECT → sCM , TCD	Understand, don't implement. Know the consistency objective and why LCM-LoRA mattered
Trajectory / step-collapsing	Progressive distillation	Read only. Historical; the idea takes ten minutes
Adversarial	ADD (SDXL-Turbo), LADD (SD3-Turbo)	Read only. GAN-flavoured; strong at ~4 steps
Teacher-free few-step	Shortcut models, MeanFlow	Read the abstracts. Frontier; notable for breaking the “you need a teacher” assumption

Current empirical picture (mid-2026): DMD2 and ADD are strong at ~4 steps. Among continuous-time consistency methods, **sCM wins at very low NFE while MeanFlow gives sharper detail at higher NFE. 1–2 steps still visibly degrades** — treat any “one-step, no quality loss” claim with suspicion until you have seen its recall number.

Four axes that actually separate the families

When choosing, ask these rather than comparing headline FIDs:

1. **Do you need a teacher?** (Everything except shortcut/MeanFlow.)
2. **Do you need real data?** (DMD2 and adversarial methods do; pure trajectory methods don't.)
3. **Do you need a discriminator?** (Adversarial and DMD2 — extra training instability.)
4. **Does it preserve the deterministic noise→sample map?** Interpolation, editing, seed reproducibility and A/B testing all depend on it. Distribution-matching and adversarial methods generally do **not**.

How to judge any distilled model — never FID alone. Adversarial and distribution-matching methods buy sharpness by shedding mode coverage, so FID can *improve* while the model quietly collapses onto a subset of the data. **Always report Recall or Coverage alongside FID, plus wall-clock NFE** (Part VIII). A distilled model that wins on FID and loses 30 % of recall is not a faster model; it is a different, narrower one.

Guidance distillation (a separate 2×)

CFG costs two forward passes per step. Guidance distillation removes one.

```

# TEACHER: 2 passes, random guidance strength each step
w = uniform(1.0, 8.0)
v_c = teacher(x, t, caption)
v_u = teacher(x, t, "")
v_target = v_u + w * (v_c - v_u)

# STUDENT: 1 pass, and w is an INPUT
v_student = student(x, t, caption, w)      # ← w fed in like a timestep

loss = mse(v_student, v_target)

```

`w` is randomised so the student learns the **whole family** of guidance strengths, not one fixed setting. At inference you pass `w` as a number and do **one** forward pass.

This is exactly what Flux-dev is — it takes a `guidance` input for precisely this reason.

The two-stage arithmetic

Base:	25 steps × 2 (CFG)	=	50 forward passes
Guidance-distilled:	25 steps × 1	=	25
+ Step-distilled:	1 step × 1	=	1 ← 50× total

Ordering — two patterns, combined is now standard

Pattern A (sequential):	base → guidance-distil → 25 fwd → step-distil → 1 fwd
Pattern B (combined):	base → one distillation absorbing both → 1–4 fwd

Pattern B is what DMD2 / LADD / Turbo actually do — the student learns to reproduce the *guided, multi-step* teacher in few steps, both wins in one run.

The Flux family makes this concrete:

Model	Guidance-distilled	Step-distilled	Steps
Flux-dev	yes (<code>guidance</code> input)	no	~20–50
Flux-schnell	yes	yes	1–4

Inference after distillation — no solver

This is where people most often go wrong.

```
# ---- 1 STEP: no loop at all ----
x = randn(...)
image = vae.decode(model(x, t_max, prompt))    # one forward pass. Done.

# ---- 4 STEPS: a loop, but NOT integration ----
x = randn(...)
for t in [999, 749, 499, 249]:
    x0 = model(x, t, prompt)                    # jump straight to a clean estimate
    if t != 249:
        x = add_noise(x0, sigma(next_t))        # corrupt partway back down
```

The model's output changed meaning.

```
Base model      → a VELOCITY (a direction to move)
Distilled model → an IMAGE ESTIMATE (the finished answer)
```

So a solver step `x + model(x)*dt` would be adding an *image* to a noisy latent scaled by a tiny number. Semantically meaningless — hence garbage.

A solver *integrates*: each step accumulates onto a path. This does not accumulate. Each iteration independently guesses the final answer, then deliberately corrupts it so the next iteration has something to improve. Guess → smudge → guess better.

Two gotchas that bite everyone

1. **CFG is usually baked in.** Distillation typically reproduces a *guided* teacher. Set `guidance_scale = 1.0`. Applying normal CFG on top produces fried output.
2. **Timesteps are fixed, not free.** Unlike a base model you cannot dial `num_inference_steps` freely — the schedule is part of what was distilled. LCM at 8 steps often looks *worse* than at 4.

Reflow, and why it lost

Reflow (from the rectified-flow paper) attacks **Source B** directly:

1. Take your trained model. Sample noise ϵ , run the full ODE, get image x_0 . Save the pair.
2. Repeat for a large set. The coupling is now **deterministic** — each noise maps to exactly one image.
3. Retrain from scratch on those pairs.

Because the coupling is a function rather than random, **the lines stop crossing**. Nothing to average → the marginal field really is nearly straight → Euler at 1–4 steps starts working. This is the basis of InstaFlow.

Terminology: “2-rectified flow” = one round of reflow; “3-rectified flow” = two rounds.

Why nobody uses it

	Reflow	Distillation
Generate millions of pairs with your own model	required	no
Full retrain from scratch	required	no (fine-tune)
Quality degrades per round	yes (self-distillation drift)	no
Can exceed the teacher	no	yes (DMD2)

2-rectified flow measurably *loses* FID at high step counts; it only wins at low step counts — and distillation delivers more few-step performance for less effort.

Dominated on every axis. The name survived; the technique did not.

Part V — Inference optimisation recipe

The mental model: memory-bound vs compute-bound

A GPU has two separate resources:

- **A huge army of chefs** = compute (tensor cores). H100: ~990 TFLOP/s dense bf16. Absurdly fast.
- **One narrow doorway to the pantry** = memory bandwidth. H100: ~3.35 TB/s. Fast, but *relatively* much slower.

MEMORY-BOUND = the chefs stand idle, waiting for ingredients
COMPUTE-BOUND = ingredients pile up; the chefs are the limit

You want **compute-bound**. That means the expensive hardware is actually working.

The roofline number

$$\text{compute} \div \text{bandwidth} = 990\text{e}12 / 3.35\text{e}12 \approx 300 \text{ FLOPs per byte}$$

This is the machine’s break-even point — the “ridge”.

your work needs < 300 FLOP/byte → MEMORY-BOUND
your work needs > 300 FLOP/byte → COMPUTE-BOUND

Why batching changes which one you are

Weights are carried through the doorway once per batch, not once per item.

Batch = 1: haul 10 GB of weights → cook 1 dish. Chefs idle. MEMORY-BOUND.
Batch = 128: haul the same 10 GB → cook 128. Chefs busy. COMPUTE-BOUND.

Critically: diffusion is already compute-bound

This is where LLM intuition misleads people.

	LLM generating text	Video / image diffusion
Items per forward pass	1 token	thousands of patches
Weight read amortised over	1 thing	thousands of things
Matmul shape	skinny	fat
Bottleneck at batch = 1	memory bandwidth	compute

An LLM writing one word at a time genuinely processes *one* item, so batch=1 is catastrophically memory-bound. A diffusion model processes the **entire frame at once** — thousands of patches through every layer. Those thousands act like a batch.

Consequence: since you are compute-bound, going faster means doing less math.

Lever	Savings	Why
1. Fewer steps (distillation)	25×	You were running the whole model 25 times
2. Fewer tokens (VAE compression)	up to 16× on attention	Attention is quadratic in tokens
3. Fusion + fewer launches	~5–20 %	Real, but smallest

The complete map: ten categories of inference optimisation

Quantisation, distillation and kernel fusion are the three you hear about, and they are genuinely the core — but they are three of ten, and they attack **different layers**:

- **Quantisation** — store and compute with fewer bits.
- **Distillation** — use a smaller model, or take fewer steps.
- **Kernel fusion** — execute the *same* model more efficiently.

The rest of this Part drills into the ones that matter for diffusion. This section is the map, so you can see what you are *not* using and decide whether that is a mistake. The **Diffusion?** column is the important one — several of these are LLM-serving techniques that do nothing for a diffusion model, and several people waste weeks discovering that.

1. Reduce precision — quantisation

BF16/FP16 → FP8 / INT8 / INT4 / FP4 . FP16→INT4 cuts raw weight storage ~4× (before metadata and scale overhead).

Applies separately to **weights**, **activations**, and (for LLMs) the **KV cache**. **Weight-only** quantisation mainly reduces memory *traffic*; quantising **activations too** is what unlocks lower-precision hardware math — and carries most of the accuracy risk.

Buys: lower weight memory, lower bandwidth, potentially faster matmul, larger batches / longer context, possibly fewer GPUs.

2. Reduce the amount of work — distillation

large teacher → smaller student , cutting parameters, layers, FLOPs, latency and cost. For diffusion and flow models it takes a *different and better* form: cut the **number of model calls**, 50 steps → 4 steps .

This is the step-function lever. Removing 90 % of the steps beats making each step 20 % faster, and nothing else on this list competes with it for diffusion.

3. Remove unnecessary parameters — pruning and sparsity

W → W_sparse . The distinction that decides whether it is real:

Kind	What is removed	Speedup?
Unstructured	individual weights → zero	Often none. The model is <i>mathematically</i> sparse but GPUs still want large dense matmuls
Structured	whole blocks, channels, heads, layers, or fixed patterns (e.g. 2:4)	Yes — hardware can exploit it (cuSPARSElt supports structured sparse matmul)

Usually needs retraining or fine-tuning to recover quality. **Rarely the first move for diffusion.**

4. Execute the same computation better — compilation and fused kernels

Naive: `matmul → write → read → bias → write → read → activation → write`. Fused: `matmul + bias + activation → one kernel`.

Cuts launch overhead, intermediate memory traffic, synchronisation, temporary tensors and idle GPU time. `torch.compile`, TensorRT, Triton, fused attention, fused normalisation and CUDA Graphs all live here.

Buys latency and utilisation — not weight memory. And critically: **it is the only category on this list that is mathematically quality-neutral.** It changes *how* the arithmetic runs, never what is computed. When you cannot afford to spend quality, this is where you start.

5. Optimise attention and model state

Technique	What it does	Matters for
KV caching	Store past keys/values instead of recomputing	LLM decode; streaming video (Part VI)
Paged KV cache	Allocate KV in blocks, not one contiguous slab → less fragmentation, more concurrent sequences	LLM serving (vLLM, TensorRT-LLM)
Prefix caching	Reuse the computed KV state for a shared prefix (e.g. one system prompt across requests)	LLM serving

Mostly an LLM-throughput category — **except** that a causal, streaming world model is an autoregressive decoder, so KV caching is central to Part VI and paged/prefix caching become relevant the moment you serve many concurrent streams.

6. Produce output in fewer sequential steps

Distinct from shrinking the network — this attacks the *dependency chain*.

- **Speculative decoding (LLM):** a small draft model proposes several tokens, the large model verifies them in one pass. Accepted tokens cost fewer expensive sequential calls.
- **Fewer diffusion steps:** `30 model calls → 4`, via better solvers, consistency models, progressive distillation, LCM, rectified flows, or few-step students.

For anything with iterative sampling — diffusion, flow policies, robot action chunks — this is usually the single largest latency win, because the iteration is the cost.

7. Batch requests better

1 request at a time → B requests per batch. **Static** batching waits for a fixed batch; **dynamic** / **continuous** batching merges requests arriving at different times and admits new ones as others finish.

Buys throughput, utilisation and cost-per-request. Costs activation/KV memory and possibly added waiting latency — and **it does not improve the latency of a single isolated request**.

⚠ **The diffusion caveat, and it is a big one.** Batching is transformative for LLMs because it escapes the memory-bound regime. Diffusion is **already compute-bound at batch=1** (see above), so there is no idle capacity to reclaim — batching gains are small. This is why real-time video costs roughly *a whole GPU per concurrent stream*.

8. Split the model across GPUs

Kind	Split	Buys	Costs
Tensor parallel	large matmuls across GPUs	aggregate memory, sometimes lower latency	inter-GPU communication
Pipeline parallel	different layers on different GPUs	very large models fit	pipeline bubbles + comms
Data parallel	full copy per GPU, different requests	throughput	no memory reduction per GPU

Parallelism distributes memory. It does not reduce total model size. Worth repeating, because it is a common misreading of “we sharded it, so it’s smaller.”

9. Offload weights or state

Move data to CPU RAM, NVMe, or another GPU (DeepSpeed-style). Buys: the model runs at all despite exceeding GPU memory, on cheaper hardware. Costs: transfer latency, lower throughput, PCIe/network bottlenecks.

This is a make-it-fit technique, not a make-it-fast technique. Never reach for it first.

10. Change the architecture

Sometimes the biggest win is designing a cheaper model up front: fewer layers, smaller width, **grouped-query / multi-query attention**, smaller vocabulary or resolution, token pruning, shorter action-chunk horizon, mixture-of-experts, **latent-space rather than pixel-space diffusion**.

An MoE has large total capacity with low *active* compute — but it does not reduce total weight memory unless experts are sharded or loaded selectively.

Architecture changes can beat every kernel optimisation combined. They also require training. **This is the same insight as the hardware co-design loop in Part VI** — latency is welded into the shape of the network, and shape cannot be fine-tuned.

The practical map

Method	Weight memory	Activation / KV memory	Single-request latency	Throughput	Needs training
Quantisation	Strong	Sometimes	Often improves	Often improves	Sometimes
Distillation	Strong	Strong	Strong	Strong	Yes
Pruning / sparsity	Possible	Sometimes	Only with the right kernels	Possible	Usually
Fusion / compiler	Little	Sometimes	Improves	Improves	No
KV-cache optimisation	None	Strong	Sometimes	Strong	No
Speculative decoding	None	Slight increase	Improves	Improves	Draft model
Batching	None	Uses more	May worsen	Strong	No
Tensor parallelism	Splits	Splits	Can improve	Can improve	No
CPU / NVMe offload	Reduces on GPU	Reduces on GPU	Worsens	Worsens	No
Fewer diffusion steps	None	Sometimes	Strong	Strong	Often
Architecture redesign	Strong	Strong	Strong	Strong	Yes

The five-verb summary

Make it smaller : quantise, prune, distil
Do less work : distil, fewer steps, speculate
Run work better : compile, fuse, optimised attention
Serve work better : batch, cache, route
Make it fit : parallelise or offload

Typical full stacks:

Workload	Stack
Large LLM	quantisation + fused kernels + paged KV cache + continuous batching + speculative decoding + multi-GPU parallelism
Diffusion / flow policy	few-step distillation + quantisation + compiled/fused attention + image or action batching
Real-time streaming video	few-step causal distillation + KV-cache streaming + fused attention + CUDA Graphs + fp8 — and one GPU per stream

Kernels, fusion, and launches

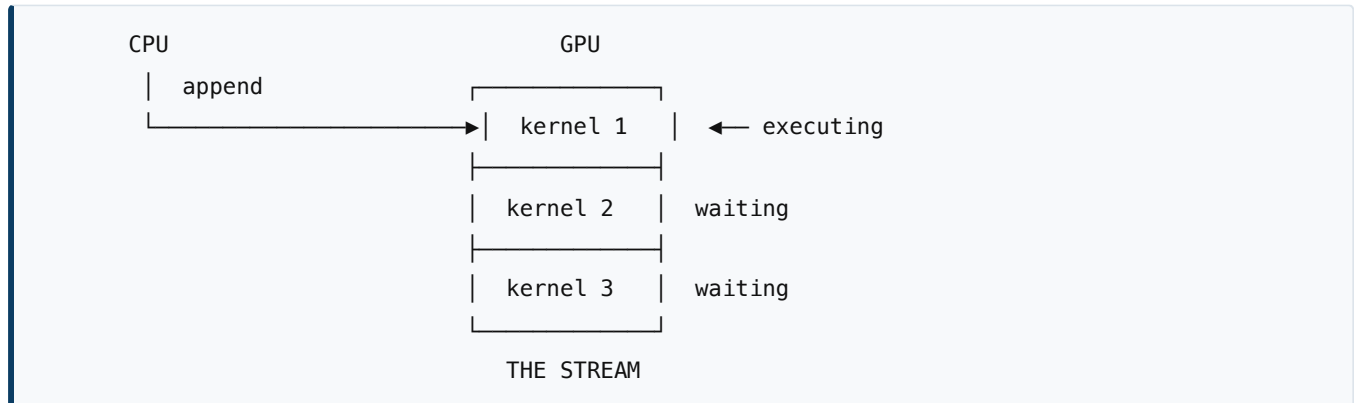
What a kernel is

One program the CPU hands to the GPU: “multiply these matrices”, “normalise this tensor”.

CUDA streams — the foundation

CPU and GPU are separate processors. The CPU cannot *run* GPU code; it **submits work**.

A stream is a queue.



Two rules:

WITHIN one stream: strictly ordered. Item N completes COMPLETELY before item N+1 starts. Never overlap.
BETWEEN streams: no ordering. May overlap freely.

PyTorch uses one stream by default. Every op goes into it, in the order you wrote it. **Order = your line order.** No hidden reordering.

The launch is asynchronous — and this defines “CPU-bound”

When the CPU launches a kernel it does **not** wait. It appends to the queue and returns immediately ($\sim 5 \mu s$).

GPU work per kernel = $500 \mu s$, launch = $5 \mu s$
CPU queues far ahead. Queue never empties. GPU 100 % busy.
→ launch cost fully HIDDEN

GPU work per kernel = $3 \mu s$, launch = $5 \mu s$
CPU cannot keep up. Queue DRAINS. GPU finishes and waits.
→ CPU-BOUND. You bought an H100 to watch it idle.

The waiter and the cook. The CPU writes order tickets onto a rail; the GPU takes them one at a time in order. If dishes take 10 minutes, the waiter stays ahead easily. If dishes take 3 seconds, the cook stands waiting for tickets. **The kitchen is not slow — the ordering is.**

On an `nsys` timeline this is literally visible as white gaps between kernels. Gaps = GPU waiting for CPU.

Fix: `torch.compile` + CUDA graphs.

Does sequential execution cost you?

With multiple streams GPUs *can* run kernels concurrently, but for a large model it buys nothing:

A matmul over 4096 tokens **already occupies every SM**. There is no idle hardware for a second kernel.

Big kernel:  all SMs busy → concurrency gains ZERO
Small kernel:  GPU underused → concurrency COULD help

So sequential execution costs almost nothing for large models. The only real loss is the launch gaps — which is exactly why launch overhead is 4 % on a DiT and 33 % on a small CNN.

The benchmarking bug you will hit

```
# WRONG – measures the CPU launch loop, not the GPU
t0 = time.time(); out = model(x); print(time.time() - t0)

# CORRECT
torch.cuda.synchronize()
t0 = time.time()
out = model(x)
torch.cuda.synchronize()
print(time.time() - t0)
```

Related: `.item()`, `.cpu()`, and `print(tensor)` each force an implicit sync. In a hot loop they drain the queue and idle the GPU.

Fusion is loop interchange

A transformer layer is not one big matmul:

matmul → add bias → layernorm → scale → activation → matmul → add residual

These are **sequential and unorderable** (each needs the previous output). But unfused, each round-trips through slow memory:

```
matmul   : read HBM, compute, write 100 MB
add bias : read 100 MB, add, write 100 MB      ← almost no math!
layernorm: read 100 MB, normalise, write 100 MB ← almost no math!
scale    : read 100 MB, multiply, write 100 MB ← almost no math!
```

Fusion does not reorder them. It swaps the loop nesting:

UNFUSED – outer loop over OPS:

```
for op in [norm, scale, activate]:  
    for elem in all_100M_elements:      ← parallel, fast  
        compute  
    write all 100M to HBM                ← SLOW, and 3 times over
```

FUSED – outer loop over ELEMENTS:

```
for elem in all_100M_elements:          ← parallel, fast  
    for op in [norm, scale, activate]:   ← all three in a REGISTER  
        compute  
    write once to HBM                    ← SLOW, but only ONCE
```

Why this is legal: these ops are **elementwise**. Element 5's computation does not need element 6's value, so the whole chain can complete for element 5 while it sits in a register.

What cannot be fused

FUSABLE: norm → scale → shift → activation → residual
 Elementwise. Each element independent.

NOT FUSABLE: matmul → matmul
 Each output of #2 depends on MANY outputs of #1.
 You need #1 COMPLETE. A genuine barrier.

The barriers are your fusion boundaries, and this is exactly the “body vs glue” split:

```
matmul [BARRIER]                compute-bound → fp8 makes math 2× faster  
↓  
norm ┌  
scale ┤ elementwise chain        memory-bound → FUSE into one kernel  
act  └  
↓  
matmul [BARRIER]                compute-bound → fp8
```

Why more layers means more launches

Not one kernel per layer — **each layer runs the same list of ops**, and each op is a kernel:

ONE DiT layer, unfused:

```
layernorm, scale, shift, qkv-matmul, rope, attention, out-matmul,  
residual, layernorm, scale, shift, mlp-up, activation, mlp-down, residual  
≈ 15 kernels
```

20 layers × 15 = 300 launches

60 layers × 15 = 900 launches

Kernels = layers × ops-per-layer. Width does not change the count at all — a wider layer is the *same* 15 kernels with bigger matrices.

Depth multiplies overhead; width multiplies useful work.

For **LLM decode at batch=1** this makes shallow-and-wide substantially faster. For **diffusion it is a modest 5–15 %** — you are compute-bound and each layer has thousands of tokens of parallel work to hide latency behind. Real, but second-order.

CUDA Graphs

```
NORMAL:      CPU tells GPU "run kernel 1" ... "run kernel 320"
              320 separate handoffs, every forward pass, 24× per second

CUDA GRAPH:  STARTUP (once):
              run the model once while CUDA RECORDS the exact
              sequence, arguments, and MEMORY ADDRESSES
              EVERY CALL AFTER:
              graph.replay()      ← ONE call. GPU runs all 320 itself.
```

Instead of dictating a 320-item checklist over the phone every frame, you mail the printed checklist once and say “run it”.

Why fixed shapes are required: the memory addresses are **baked into the recording**. If a tensor changes size it gets a different allocation at a different address, and the recording points at the wrong memory. Hence fixed-size ring buffers for KV caches — allocate once at startup, addresses never move, the recording stays valid forever.

FlashAttention

It loads small tiles continuously — a streaming algorithm. Nothing is ever fully resident.

```
for each Q tile (128 tokens):
  LOAD Q tile → SRAM
  init running_output, running_max, running_sum in SRAM

  for each K,V tile (128 tokens):
    LOAD this K,V tile → SRAM      ← NEW LOAD each iteration
    compute the 128×128 score block ← entirely in SRAM
    update the running values      ← ONLINE SOFTMAX
    DISCARD the tile

  WRITE this Q tile's output → HBM
```

The clever part is the math, not the loading. Softmax normally *cannot* be computed incrementally — you need the max and sum over the entire row, which is what forces materialising the full $N \times N$ matrix. **Online softmax** maintains a *running* max and sum, retroactively rescaling accumulated values when a larger max appears. **Mathematically exact — not an approximation.**

This is the same loop interchange as fusion, applied to the one place where naive softmax appeared to forbid it.

The numbers (one head, N = 6240, d = 64, bf16)

The N×N score matrix: 6240 × 6240 × 2 bytes = 77.9 MB
 ↑ an INTERMEDIATE nobody wants; it exists
 only so softmax can consume it

The irreducible data: Q, K, V, O = 4 × 6240 × 64 × 2 ≈ 3.2 MB

	Standard	FlashAttention
Real data (Q,K,V,O)	3.2 MB	3.2 MB
Score matrix traffic	~312 MB (write, read, write, read)	0 MB
Total HBM traffic	~315 MB	~3.2 MB
FLOPs	10 GFLOP	10 GFLOP — identical
Arithmetic intensity	32 FLOP/byte	~3000 FLOP/byte
vs. the 300 ridge	below → MEMORY-BOUND	above → COMPUTE-BOUND
Tensor core usage	~11 %	~100 %

(K+V together are only 1.6 MB, which fits in the H100’s 50 MB L2 cache — so per-tile re-reads are served from cache, not HBM.)

98 % of standard attention’s memory traffic is moving a matrix nobody wants. FlashAttention deletes it.

What actually happened: it did not make the math faster. It removed the traffic that was starving the math.

Calibrate expectations: real measured speedups are 2–4× (up to ~7× at long sequence lengths), not the idealised ratio. Two reasons: real baselines already avoid materialising the full matrix, and real kernels achieve 40–70 % of peak FLOPS.

And this closes the story

Once attention is compute-bound, memory optimisation is finished. The only remaining levers are doing less math:

- fewer tokens (VAE compression) ← quadratic
- sparser attention (spatial/temporal factorisation)
- lower precision (fp8 – moves the ridge AND doubles the math)

This is why the VAE compression ratio is the highest-leverage architecture number you pick.

Token count and the quadratic trap

```
480 × 832 pixels
  ↓ 8× compression
60 × 104 latents = 6,240 tokens
```

Two effects, and the second is brutal:

- 1. **Everything scales with token count** — linear.
- 2. **Attention scales with tokens squared** — quadratic.

```
4× compression: 120 × 208 = 24,960 tokens → attention ∝ 623M pairs
8× compression:  60 × 104 =  6,240 tokens → attention ∝  39M pairs

4× fewer tokens → 16× less attention work
```

But modern models factorise attention

FlashAttention does NOT reduce the FLOPs. It is still $O(N^2)$ compute; it only reduces memory traffic.

Technique	Complexity	Exact?
FlashAttention	still N^2 compute	exact
Windowed / local	$N \cdot k$ — linear	approximation
Factorised spatial/temporal	see below	exact , different connectivity
Linear attention / SSM	N	quality cost

Video models really do factorise — separate spatial attention (within a frame) from temporal attention (same position across frames):

```
Full:      (F × S)² = F²S²
Factorised: F · S² + S · F²
```

For 20 frames × 6,240 spatial tokens that is a **~20× reduction, exact**.

The VAE point survives, refined. With factorisation the quadratic applies to **spatial tokens per frame**:

	tokens/frame	spatial attn	temporal attn	matmuls
8×	6,240	1×	1×	1×
4×	24,960	16×	4×	4×

Quantisation

WEIGHT-ONLY:

store weights in 8 bits ← saves memory reads

dequantise to bf16 before the matmul

matmul runs at bf16 speed ← MATH IS NOT FASTER

WEIGHT + ACTIVATION (fp8):

store weights in 8 bits ← saves memory reads

keep activations in 8 bits

matmul runs in fp8 ← MATH IS 2× FASTER on H100

	Bottleneck	Weight-only	Weight + activation
LLM decode	memory	big win	big win
Diffusion	compute	~nothing	~2× — use this

Why weight-only does nothing for diffusion: you were not waiting on memory. Making the thing you were not waiting for smaller changes nothing; the matmul still runs at bf16 speed, and the matmul was the bottleneck.

Why LLM people default to weight-only: LLM decode *is* memory-bound. It is the right answer for their problem, not yours.

What to quantise, and what never to

Component	Precision	Why
DiT / transformer blocks	fp8	90 %+ of compute. Near-lossless here.
Final output projection	bf16	Small; errors go straight to the image
Timestep / guidance embedding MLPs	bf16	Tiny, and they modulate <i>everything</i>
LayerNorm / adaLN	bf16	Cheap and numerically sensitive
First & last transformer block	bf16	Commonly kept high-precision; low cost
VAE decoder	bf16 / fp16	fp8 here produces visible artefacts. Do not.
Text encoder	bf16	Runs once. No reason to risk it.

Keeping ~10 % of the model in bf16 costs almost nothing and removes most of the risk. This is what production fp8 setups do.

Quality reality

Format	Impact
fp8 (E4M3), per-channel scaling	Usually indistinguishable; occasionally slight softening of fine texture
fp8, naive per-tensor scaling	Noticeable — scaling granularity matters more than the format
int8	Needs calibration; more visible degradation
int4	Visibly bad for images. Fine for LLM weights, not this

The failure mode to know: DiT adaLN modulation can produce activation outliers with large dynamic range. Per-tensor fp8 scaling clips them. **Per-channel or block-wise scaling fixes it.** If fp8 looks bad, check scaling granularity before blaming the format.

Always A/B on 20+ fixed seeds and look with your eyes. Aggregate metrics hide this.

torch.compile vs TensorRT vs ONNX

	Type	When it runs	Runtime needs	Output
torch.compile	JIT compiler inside PyTorch	first call	Python + PyTorch	in-memory compiled callable
ONNX	file format — not a compiler, not a runtime	n/a	n/a	model.onnx
TensorRT	AOT compiler + runtime	offline build	no Python required	.engine binary

Path 1 (easy):

PyTorch

→ torch.compile →

fast PyTorch

← start here

Path 2 (export):

PyTorch

→ torch.onnx.export →

.onnx

→ TensorRT

Path 3 (hybrid):

PyTorch

→ torch-tensorrt →

TensorRT (no ONNX)

You never need ONNX to use torch.compile. ONNX is only for leaving PyTorch. torch-tensorrt now skips it, and works as a torch.compile backend:

```
model = torch.compile(model, backend="tensorrt")
```

Why torch.compile can't get the last 30 %

1. It expresses your algorithm efficiently — it will not *replace* your algorithm. Write naive attention and compile it: Inductor will fuse the scale into the matmul epilogue and tidy the softmax. **It will not invent FlashAttention.**

Proof by history: FlashAttention — the single largest kernel win in the field — was hand-written by a human. No compiler found it.

2. Matmuls come from a fixed template library. Inductor calls cuBLAS or autotunes among a **preset menu** of Triton templates. It searches within a vocabulary; hand-writing lets you extend the vocabulary.

3. Graph breaks. Data-dependent branches (`if x.max() > t`), `.item()`, unsupported ops, dynamic shapes — each splits the graph. At every break you fall back to eager and lose fusion *and* CUDA-graph capture across the boundary.

4. It must preserve exact semantics. A human can decide “fp8 is fine here”, “I can reuse this buffer”, “skip this epsilon guard”. A compiler cannot take those risks.

5. It does not know your deployment invariants. Batch is *always* 1, sequence is *always* 6,240, dtype is *always* bf16. You specialise and delete every guard; the compiler keeps them.

Where each is best practice

Serving on GPUs, in Python, model changes sometimes

→ `torch.compile`. ~95 % of cases.

Model FROZEN, shapes FIXED, and you need the last ~20 %

→ TensorRT (via `torch-tensorrt`; skip ONNX)

You must run WITHOUT Python – C++ service, Jetson, embedded

→ TensorRT. Its strongest remaining case.

Non-NVIDIA target, mobile, ONNX Runtime, C#/Java

→ ONNX. Requirement-driven, not performance-driven.

Where TensorRT actually wins

Not on DiTs. The key fact: **launch overhead is a fixed cost per kernel regardless of work done.**

BIG kernel: $500\ \mu\text{s work} + 5\ \mu\text{s launch} = 1\ \% \text{ overhead}$ → irrelevant

SMALL kernel: $3\ \mu\text{s work} + 5\ \mu\text{s launch} = 165\ \% \text{ overhead}$ → catastrophic

ResNet-50 batch 1: ~150 low-work kernels → launch cost ~33 % of runtime

TensorRT fuses conv+BN+ReLU (150→50) then graphs (50→1)

→ BIG WIN

A DiT, 4096 tokens: ~320 kernels, each hundreds of μs of work

$320 \times 5\ \mu\text{s} = 1.6\ \text{ms}$ of 40 ms → 4 %

→ SMALL WIN

Strong TensorRT territory	Why
CNNs / UNets (SD1.5, SDXL, ResNet, YOLO)	Dozens of conv shapes; fusion + per-layer autotuning pays off
Small models at batch=1	Launch overhead dominates
INT8 with calibration	Best-in-class calibration tooling
Jetson / embedded	Realistically the only option
C++ / no-Python serving	Its strongest case

TensorRT’s superpower is optimising across many small operations. A DiT has few, enormous operations.

Speed reality

eager	1.00×
torch.compile	~2.0×
TensorRT	~2.2–2.5×

The gap nearly vanishes on DiTs — both end up calling the same cuBLAS/CUTLASS matmuls and the same FlashAttention kernel, and that is 90 % of the time. For a DiT the remaining ~10–20 % rarely justifies rebuild-per-GPU, rebuild-per-shape, and opaque debugging.

Profiling

The three-tool ladder

Tool	Question it answers	When
torch.profiler	<i>Which module/op eats my time?</i>	always, first
nsys	<i>Are there gaps? Is the GPU idle? Am I CPU-bound?</i>	when time does not add up
ncu	<i>Is this kernel memory- or compute-bound?</i>	when choosing an optimisation

```
with torch.profiler.profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True, with_modules=True, with_flops=True,
) as prof:
    model(x)

print(prof.key_averages().table(sort_by="self_cuda_time_total", row_limit=25))
```

Gives you: time per operator and per module, call counts, shapes, CPU-vs-GPU split. **This is where you find “the VAE is 30 % of my runtime”.**

It does NOT give arithmetic intensity:

```
torch.profiler:  time  ✓  
                  FLOPs ~ (with_flops=True – estimates matmul/conv only)  
                  BYTES x   ← the missing half of the ratio
```

For intensity use **Nsight Compute**:

```
ncu --set full -o profile python generate.py  
ncu --section SpeedOfLight_RooflineChart python generate.py
```

Per kernel it reports DRAM bytes read/written, achieved FLOP/s, % of peak memory and compute throughput — **and draws the roofline chart with your kernel plotted on it.**

nsys shows the timeline. White space between kernels means you are CPU-bound — Python is the bottleneck, and `torch.compile` + CUDA graphs is the fix, not kernel work.

The shortcut you can usually take

Op	Verdict
Big matmuls, thousands of tokens	compute-bound
LayerNorm, activations, adaLN modulation, residuals	memory-bound
Attention with FlashAttention	compute-bound
Attention without it	memory-bound

Which gives the answer you needed: **the transformer body is compute-bound → fp8 weights *and* activations; the glue is memory-bound → fuse it.**

Or by hand:

```
FLOPs ≈ 2 × params × tokens  
bytes ≈ weight bytes + activation bytes  
intensity = FLOPs / bytes      vs. ~300 for H100 bf16
```

Crude, but usually enough to make the fp8-vs-weight-only call correctly. **Reach for `ncu` when a specific kernel surprises you**, not routinely.

The ordered optimisation recipe

Rule 0 — profile before touching anything

Find where the milliseconds actually are. **Typical DiT: 85–95 % transformer, 3–8 % VAE, 1–2 % text encoder.** If the VAE is eating 30 %, your problem is not the transformer.

Phase 1 — free wins (an afternoon, no training, no quality risk)

1. Verify SDPA / FlashAttention is active (default in PyTorch – confirm)
2. bf16 everywhere (if somehow in fp32)
3. `torch.compile(model, mode="max-autotune")` ~2×
4. `channels_last` (if any conv / UNet)
5. Warm up before measuring (first call includes compile time)

Checkpoint: measure. Fast enough? Stop.

Phase 2 — schedule and steps (a day, no training)

6. Karras / shifted schedule ← bigger win than solver choice
7. Correct solver: Euler (FM) | DPM++ 2M (diffusion)
8. Binary-search step count DOWN:
50 → 30 → 25 → 20 → 15, comparing on FIXED SEEDS 2–3×
9. Tune CFG scale – sometimes lets you drop steps further

Checkpoint: $\approx 4\text{--}6\times$ cumulative, zero training, zero quality loss. Most teams are done here.

Phase 3 — precision (a few days, small quality risk)

10. fp8 (E4M3) weights AND activations on the transformer body
11. KEEP bf16: VAE, text encoder, final projection, embedding MLPs, norms, first/last block
12. Per-channel or block-wise scaling – NOT per-tensor
13. A/B on 20+ fixed seeds, look with your eyes 1.5–2×

Checkpoint: $\approx 6\text{--}10\times$ cumulative. Still no training.

Phase 4 — token count (needs retraining, or accept lower resolution)

14. Lower output resolution if the product tolerates it
15. Higher-compression VAE ← requires retraining the DiT
16. Factorised spatial/temporal attention ← architecture change
up to 4–16× on attention

Phase 5 — distillation (weeks, real quality cost)

17. Guidance distillation → w becomes an input, 1 pass 2×
18. Step distillation:
 - LCM-LoRA – cheapest (~32 A100-hr), blurriest
 - DMD2 – best quality, can beat the teacher, heaviest
 - LADD – sharpest, some diversity loss→ 4 steps 5–6×
- 1 step 4× more

Checkpoint: $\approx 70\text{--}100\times$ **cumulative**. Accept: reduced diversity, fixed step count, baked-in guidance.

Phase 6 — only if latency is your product

19. TensorRT / custom runtime	1.2×
20. Hand-written fused kernels	1.3–2×
21. Causal architecture (streaming)	← enabler, not a speedup
22. Hardware co-designed architecture	← train from scratch

Months, a specialist team. Do not start here.

By latency target

Target	Do
● Seconds (offline, batch, research)	<code>torch.compile</code> . Stop.
● 1–3 s (interactive images)	Phases 1–3. $\approx 6\text{--}8\times$, no training . Sweet spot for effort/reward.
● <500 ms (chat-speed)	+ guidance distillation + 4-step distillation. $\approx 70\text{--}100\times$. Weeks.
● <40 ms (real-time video)	+ 1-step + causal architecture + high-compression VAE + custom kernels. Months.

The one-line rule

Phases 1–3	$\approx 6\text{--}10\times$	~ 1 week, no training	← DO ALL OF THIS
Phase 4–5	$\approx 10\times$	$\sim 1\text{--}2$ months, quality cost	← only if needed
Phase 6	$\approx 1.5\text{--}2\times$	~ 6 months, a team	← only if it IS the product

If you have not reduced your step count, do not write a CUDA kernel. You would be chasing 4 % while leaving 250 % untouched.

Why Tier 6 is “expensive”

Engineering time, not money.

TensorRT: export fails on unsupported ops; engine build takes minutes to hours; the `.engine` is locked to one GPU model + TensorRT version + shape profile (new GPU → rebuild); debugging is opaque with no Python stack trace.

Hand-written kernels: months of a scarce specialist’s time; change one layer → rewrite; new GPU generation → retune or rewrite; **you own correctness forever.**

Part VI — Real-time video

Why offline models cannot stream

Two independent barriers, and **only one is fixable by distillation**.

Barrier 1 — step count

A base model needs ~20–30 solver steps. At 5 B parameters over 81 frames of latents that is seconds-to-minutes per clip, not 40 ms. **This one distillation fixes.**

Barrier 2 — it is not a streaming model

A batch video model denoises **all frames jointly**, with bidirectional attention across the whole clip. Frame 1 attends to frame 81. Therefore:

It cannot emit frame 1 until it has finished computing frame 81.

Your latency floor is the entire clip, no matter how fast the hardware. **Even with infinite compute and 1-step sampling you could not stream it.** Distillation does not help; the dependency structure is wrong.

A real-time model must be causal: generate frame t conditioned only on frames $< t$ plus the live input. That is a different training setup, not a different inference config.

Drift — the central research problem

A streaming model conditions on its own previous outputs, creating a feedback loop with a poisonous mismatch:

History the model sees	
During training	real frames from a real video — clean
During inference	its own outputs — containing small errors

The model was never trained on its own slightly-wrong output, so it does not recognise an artefact as an error. It treats it as **content** and faithfully continues it. This is **exposure bias**:

```
frame 1:  tiny error (slightly off colour)
frame 2:  inherits it + reinforces it + adds a new error
frame 3:  builds on two errors, both treated as ground truth
...
frame 300: unrecognisable
```

The cruel irony: temporal consistency — the thing that makes video look smooth — is exactly what makes drift unrecoverable. The model is *rewarded* for continuing whatever it sees in its history, so it faithfully amplifies its own mistakes. There is **no attractor at “correct”** — it is a random walk with positive feedback.

Symptoms: colours progressively saturate or wash out; detail smears; a face slowly stops being that person; the scene morphs into texture soup, then freezes.

Naive autoregressive T2V: fine for 1–3 s, visibly degraded by 5–10 s, collapsed by 20–30 s. **A product needs unbounded.**

Why pure text-to-video cannot escape

Frame *t* is conditioned on exactly two things: the **text prompt** (global, weak — it says nothing about the correct pixels of frame 500) and **its own history** (the contaminated signal).

The loop is closed. No signal from outside it. Nothing can ever tell the model it has drifted.

Why video-to-video breaks the loop

Frame *t* is also conditioned on **input frame *t***, arriving fresh from the camera. That frame is ground truth, **from outside the feedback loop**, and uncontaminated by any previous model error.

Every frame gets a fresh, correct anchor — geometry, pose, layout, motion, lighting — **re-injected every 40 ms**. The model’s remaining job shrinks to *apply a style transformation to this given frame*, much closer to a per-frame function than a generative rollout.

T2V = walking blindfolded by dead reckoning. Small heading errors accumulate; after 500 steps you are nowhere near target and cannot even know. **V2V** = walking with your eyes open. You wobble, but you re-observe the actual path every step. **Error can never accumulate past one step’s worth.**

In control terms, the source video is a **setpoint**. You have moved from open-loop drift to a closed loop with a reference signal.

Drift is relocated, not eliminated

You still feed history in V2V (otherwise you get flicker — the knight’s armour would have different rivets every frame). But look at *what* the feedback path carries:

Channel	Source	Can it drift?
Structure, motion, layout, pose	input frame, every frame	No — externally re-supplied
Style intensity, invented detail	history	Yes, slowly

The castle might get progressively more orange. **Cosmetic.** Structural collapse cannot happen, because structure is never the model’s to remember.

V2V confines drift to the channel where it is cosmetic and eliminates it from the channel where it is fatal.

The fixes — the forcing family, in historical order

Every technique below attacks the same feedback loop, and they get progressively less ad-hoc. Read the table as one argument.

Technique	Mechanism	What it fixes	Cost
Teacher forcing	Condition on ground-truth history	Nothing — this is the problem	Maximum train/test mismatch
History augmentation (<i>a.k.a. noise-augmented context</i> — GameNGen, MirageLSD)	Inject synthetic corruption into the history as data augmentation, and tell the model the level	The first fix. The model learns to work from a <i>corrupted</i> history, so its own flawed output is in-distribution — it can spot and correct rather than propagate	Cheap — ordinary step cost
Diffusion forcing	Independent noise level per frame	The principled version of the row above, with a variational bound over all subsequences. Rollouts extend past the training horizon where baselines diverge ; also unlocks variable-length generation and new guidance schemes	Cheap; but never literally simulates inference
Self-forcing	Roll out the student’s own generations autoregressively during training (with KV cache) and supervise with a <i>holistic</i> distribution-matching loss over the sequence	Closes the train/test gap directly rather than approximating robustness. Streams ~16 FPS on H100, ~10 FPS on a single 4090	Expensive — you simulate during training; sequential, so slower per step. Degrades past the ~5 s training horizon
Causal Forcing / ++	Causal ODE / consistency distillation + asymmetric DMD	Fixes Self-Forcing’s residual issues and scales ; the current production recipe	Needs a teacher; most machinery

Start with history augmentation. It is materially cheaper, it is what MirageLSD reports using, and you should measure it before paying for rollout-based self-forcing.

⚠ **These are not synonyms, and the literature muddles them.** “Causal / autoregressive diffusion” is the **category**. Diffusion forcing changes the **noising scheme**; self-forcing changes **what the model is conditioned on during training**. They are **orthogonal** — modern systems (MIRA, Matrix-Game 2.0) use both.

Diffusion forcing is not “just add noise” — the independence is the point

Ordinary video diffusion draws **one** σ and applies it to the whole clip. Diffusion forcing draws a **separate, independent σ per frame**. That sounds like a detail; it changes what the network learns:

- Standard training covers a 1-D family: (*one noise level*). Diffusion forcing covers the full **2-D grid of (frame position $\times$ noise level)** — including the asymmetric corner **clean-ish past, very noisy future**. That corner is causal generation, and standard training never visits it.
- So at inference you can **choose the noise schedule across time**, not just across steps: fully denoise the past for sharpness, or hold it at low-but-nonzero noise for robustness to your own errors. A dial that otherwise does not exist.

- It **subsumes both special cases**: all σ equal = full-sequence diffusion; past $\sigma=0$, future $\sigma=\text{max}$ = next-frame prediction. One objective, both regimes.
- **The implementation really is small**: sample σ of shape `[B, T]` instead of `[B]`, broadcast per frame, feed each frame's σ into the conditioning. This is why it is the cheapest thing on the list to retrofit.

How to measure drift — six tests, most to least useful

You will claim “less drift.” Prove it.

1. **Drift curve** — the headline plot. Roll out N steps from a real seed frame; compute FID (or LPIPS against the true continuation) **per rollout position**; plot quality against step index. Stable models flatten; unstable ones bend and then cliff. Overlay one curve per technique on **identical seeds**.
2. **Statistics over time** — track mean, std and norm of generated frames per step. Drift shows up as a slow statistical march (contrast creeping, saturation growing) **before** it is visible. Your cheap early-warning signal.
3. **Survival time** — steps until a human says “that’s broken.” Blunt, but it is what users experience.
4. **Action-response fidelity** — press left; does it go left? **Controllability degrades before appearance does**, so this catches problems the eye misses.
5. **Ground-truth state comparison** — if your simulator exposes state, withhold it from training and use it **only for evaluation**. MIRA does exactly this with Rocket League physics, so it can ask “is the ball where real physics says?” rather than “does it look plausible?” Strongest signal available.
6. **Return-to-viewpoint (loop closure)** — leave and come back, compare frames at the same pose. ⚠️ **This measures a different axis** — see Part VII.

Hardware co-design

Why batch=1 is permanent, and what it costs

You cannot batch a live stream across time. Frame $t+1$ does not exist yet.

You *can* batch across **concurrent users** — 50 streams each sending a frame at roughly the same instant. Production does this. But you can only wait a few milliseconds to collect a batch, because the whole budget is 40 ms.

And because diffusion is already compute-bound at batch=1, batching barely helps:

	LLM serving	Real-time video
Batching throughput gain	10–100×	small
Why	escapes memory-bound	already compute-bound
Cost per user	fraction of a GPU	~a whole GPU

Every concurrent video stream needs its own GPU’s worth of compute. There was no idle capacity to reclaim. That is the real reason real-time video AI is expensive, and why **the only lever with real leverage is doing fewer steps**.

What “co-designed against silicon” means

Four architecture choices that are *purely* latency decisions:

(a) **Dimensions that fit tensor-core tiles.** H100 `wgmma` operates on fixed shapes; FlashAttention kernels are specialised for `head_dim ∈ {64, 128}`. `head_dim = 80` falls off the fast path or gets padded to 128 — **you pay for 128 and use 80**. Same for hidden dim: 1536 tiles cleanly, 1500 pads and wastes the difference on every matmul in every layer.

(b) **Shallow-and-wide vs deep-and-narrow.** Width is parallel; depth is sequential. Modest for diffusion (5–15 %), large for LLM decode.

(c) **Fixed-size history window.** Static shapes let CUDA Graphs work; the kernel picks the number, not quality.

(d) **VAE compression ratio.** Quality might want 4×; the budget demands 8×. **You choose from the frame budget, then train the best VAE achievable at that ratio.**

Why a checkpoint cannot give you any of this

Every one of (a)–(d) is welded into the weights. `hidden_dim`, `head_dim`, `num_layers`, attention pattern, VAE ratio — none can change without retraining.

Fine-tuning changes the values in the weights. It cannot change the shape. And the shape is what determines latency.

You bought a cargo ship. You can swap the engine, strip the interior, repaint the hull. You cannot make it a speedboat, because the hull is wrong — **and latency is the hull**.

The co-design loop

1. Fix the budget: 40 ms/frame, batch=1
2. Microbenchmark: cycles per matmul shape / attention pattern / memory access, on the real chip, at batch=1
3. Build a simulator: architecture config → predicted latency
4. SEARCH configs: layers × width × head_dim × window × VAE ratio × step count — keep only those under 40 ms
5. Among survivors: pick the highest quality
6. Train from scratch ← unavoidable

Note which variable is the constraint. Normal practice: quality is the constraint (“must beat X on FID”), latency is optimised afterward with kernels. **Co-design inverts it** — latency is a hard wall in the search space, quality is the objective function. That inversion produces a completely different architecture.

Kernel engineering is hardware-specific

Three levels, each requiring real work:

Level	Example	Effort
Vendor	CUDA / ROCm / Metal / a custom ISA (Etched Sohu, Groq, Cerebras)	full rewrite
Architecture	A100 (Ampere) → H100 (Hopper) → B200 (Blackwell)	substantial rewrite
SKU	H100 SXM vs PCIe vs H200	retuning

Hopper introduced genuinely new primitives — `wgmma` async tensor-core ops, TMA async copies, thread-block clusters, distributed shared memory. **FlashAttention-2** → **FlashAttention-3** was substantially a Hopper rewrite using instructions that do not exist on Ampere.

The cost of the moat: every GPU generation is re-work. That is ongoing headcount.

Case study: Decart

A clean illustration of everything above.

Product	What it is	Provenance
Oasis (Oct 2024)	Real-time playable Minecraft, no game engine. ViT spatial autoencoder + DiT latent backbone, trained with Diffusion Forcing , temporal attention interleaved between spatial layers. Millions of hours of gameplay. 47 ms/frame, 360p, 20 fps on H100 via a proprietary inference framework .	Custom, from scratch. Development began 2022 — before Wan, Flux, or SD3 existed. Collaboration with Etched (Sohu, a transformer ASIC).
MirageLSD (2025)	First live-stream diffusion. Explicitly “built on a custom model called Live Stream Diffusion (LSD).” Diffusion forcing + history augmentation + shortcut distillation + model pruning + custom CUDA mega-kernels + architecture aligned to GPU hardware. ~40 ms, 20 fps, 768×432.	Custom.
Lucy 2 / 2.5	Proprietary ~14 B real-time world-transformation and image-to-video. 1080p 30 fps, ~40 ms. Distillation + hardware co-design.	Not disclosed.
Lucy Edit Dev	Open weights , 5 B, instruction-guided video editing, 480×832, up to 81 frames. Offline/batch. Non-commercial licence.	Built on Wan 2.2 5B — hence a flow-matching backbone.

The pattern

The real-time core is custom; the open-weight release is the Wan-derived one. And this split is forced, not merely strategic: you cannot reach 40 ms/1080p by fine-tuning Wan, because Wan is bidirectional and non-causal — the dependency structure is wrong at the root. Their moat is architecture co-designed against silicon,

which is precisely what you cannot inherit from someone else’s checkpoint.

Give away what is not the moat (offline edit quality on a donated backbone); keep what is (real-time inference).

The arithmetic that explains it all

At 24 fps you have 41 ms per frame, total.

Stage	~Budget
VAE encode (high-compression VAE)	~4 ms
Conditioning (text embedding cached; input frame; history)	~0
Denoise: 1–2 forward passes of the distilled DiT	~30 ms
VAE decode	~4 ms
Feed back previous outputs as history	—

30 solver steps in 41 ms → 1.4 ms per forward pass of a 14 B model
NOT ACHIEVABLE on any hardware that exists

1 step in ~30 ms → achievable, with hardware co-design

Distillation is not an optimisation for them — it is the enabling condition for the product to exist. And once you are at 1 step, DPM-Solver++, Heun, step schedules, and curvature all become irrelevant. There is no trajectory left to integrate. That is why their engineering effort went into the chip.

Build plan: converting a batch model to streaming

Prior art — read these first

Work	What it gives you
CausVid	Converts a bidirectional video diffusion model → causal autoregressive student, then DMD-distils it. Literally this project.
Self-Forcing	Fixes drift via autoregressive rollout during training. Real-time streaming on a single H100.
Diffusion Forcing	Per-frame independent noise levels → variable-horizon causal generation.
CausVid LoRA for Wan	The community already ships causal/few-step LoRAs for Wan 2.1/2.2 — same lineage.

You are not inventing this. That cuts the risk enormously.

The pipeline

Stage 0 — attention surgery. Replace bidirectional temporal attention with **block-causal**: frame-chunk t attends only to chunks $\leq t$. Add a KV cache with a **fixed-size ring buffer** (static shapes $\rightarrow$ CUDA Graphs work).

You cannot just flip the mask and infer. The learned features assume future context; zero-shot masking degrades output badly.

Stage 1 — causal adaptation fine-tune. Retrain the masked model on the original task, still multi-step. Recovers most of the lost quality. **This is the step people skip and then wonder why it looks broken.**

Stage 2 — distillation to 1–4 steps. Two options:

- **DMD2** — best quality, can exceed the teacher. But memory-heavy: student + frozen teacher + trainable fake-score net $\approx$ 130 GB+ before activations. **8×H100 minimum, FSDP**
- **Shortcut distillation** (what MirageLSD reports) — step-size-conditioned self-distillation. One model handles 1-step through N-step, **no separate fake-score network**, so far lighter on memory.

Stage 3 — fix drift. Start with **history augmentation** (cheap: inject synthetic corruption into history). Escalate to **self-forcing** (expensive: autoregressive rollouts) only if measurement demands it.

Stage 4 — inference engineering. KV cache, fp8, CUDA graphs, TensorRT. **Watch the VAE** — at 30 fps it can consume your whole budget. Wan’s VAE is already causal 3D, which helps.

What works strongly in your favour

Video-to-video anchoring (see §VI.2) bounds drift structurally. Keep the source frame as a **first-class conditioning input on every single frame**, not just at the start of a chunk. This materially reduces Stage 3’s cost versus a T2V project.

Cost estimate

5 B parameters; H100 at $\sim$ \$2–3/hr on-demand, $\sim$ \$20–25/hr for an 8× node.

Stage	H100-hours	Cost
Data generation (teacher-produced edit pairs)	500–3,000	\$1–9 k
Causal adaptation fine-tune	500–2,000	\$1–6 k
Distillation	300–1,500	\$1–5 k
Drift fix (history augmentation is the cheap path)	200–2,000	\$0.5–6 k
Successful-path subtotal	1,500–8,500	\$4–26 k
Realistic (3–5× for failed runs)	5,000–30,000	\$15–90 k

Centre of gravity: $\sim$ \$30–50 k compute. But **engineering dominates**: 2–4 person-months if someone has done video diffusion distillation before, **6–12 person-months if learning** — \$50–250 k loaded, several times the GPU bill.

Licence — resolve this first

A model distilled from non-commercially-licensed weights is a derivative work. Distilling Lucy Edit Dev does not launder its licence.

Build on Wan 2.2 directly — it is Lucy Edit’s own base and openly released. You would add the instruction-editing capability yourself, but you stay commercially clean. **Read the exact licence text yourself before committing.**

Realistic expectations

Your first target	8–15 fps at 480p on a single H100
A mature product	30 fps at 1080p, ~40 ms

That gap is mostly chip-level work. **Getting to streaming at all is the hard research step**; 10→30 fps is a grind of quantisation and kernels. Also budget for a **visible quality drop** versus the bidirectional base — causal masking costs something real, permanently.

Decision

Option	When
Use a hosted API	Zero ML risk, ships this month. Correct unless real-time restyling is your product.
Build on Wan 2.2 + CausVid/Self-Forcing	3–6 months, ~\$50 k compute, commercially clean, you own it. Correct if this is core IP
Licence from the vendor	Worth a conversation before spending 6 months.

Part VII — Interactive world models

Part VI covered streaming *video*: a stream in, a stream out. A **world model** adds a control loop — the next frame depends on what the user just *did*. That single change brings three problems that do not exist in video generation, and this Part is about those three.

What makes a world model different

	Streaming video (Part VI)	Interactive world model
Conditioning	text, or an input video frame	actions + history
External anchor	V2V has one (the camera feed)	None — the loop is fully closed
Extra failure modes	drift	drift + controllability loss + spatial amnesia
Evaluation	perceptual quality	quality + does it obey + does it remember

The absence of an external anchor is the crux. Part VI’s cleanest fix was video-to-video: re-inject a ground-truth frame every 40 ms and error cannot accumulate past one step. A world model has no such frame — the world exists only inside the network. Every mitigation in this Part is a way to manufacture the anchor that V2V got for free.

The two axes, and why conflating them wastes months

- **Temporal stability** — “does the image degrade as I roll out?” (drift; Part VI)
- **Spatial memory** — “I turned around; is the room still the *same* room?”

These are independent. A model can be perfectly stable and have **zero** memory: it never diverges, it just quietly invents a different room each time you look back. Solving one does not solve the other, and the 2026 systems each attack a *different* one. Any paper claiming “long-horizon” without saying which axis it means should be read carefully.

System	Problem it actually solves	Mechanism
MIRA	Synchronised multi-view — several players seeing the same event consistently from different cameras at the same instant	Joint generation. One model emits all 4 first-person views together, trained on 4 synchronised views per match. Consistency is implicit in the joint forward pass. No external memory
MultiGen	Persistence + editability + arbitrary player count	Explicit external state outside the network; Memory / Observation / Dynamics decomposition
Matrix-Game 3.0	Spatial consistency over minutes	Retrieval — camera-aware selection of relevant past frames, injected into joint attention

MIRA — joint generation, and diffusing in representation space

Architecture: a 5B DiT plus a 600M representation autoencoder, trained with **diffusion forcing in the codec's latent space**. Simulates 4-player Rocket League at 576p / 20 fps on a single B200. Actions are 3-valued axis commands (steer, throttle, yaw, pitch, roll) plus binary buttons (jump, boost, handbrake) at 15 Hz, with **action dropout** (Part II) giving autopilot for free.

The codec is the headline result, not the multiplayer part. MIRA replaces the VAE encoder with a **frozen DINOv3-L/16** and trains only a decoder — the RAE pattern from Part I. They report this “gave us the highest leap in modeling performance” and credit it for long-rollout stability.

State the stability claim precisely, because it is easy to overstate. MIRA **does** use diffusion forcing — that is its training paradigm, not an optional extra. The claim is that it needed **no additional specialised anti-drift machinery on top**, and that the **codec** contributed more than the forcing scheme did.

How the multiplayer works. Training data is four simultaneous, frame-aligned first-person views of one match. The model emits **all four in one forward pass**, conditioned on **all four players' actions**. There is no synchronisation *step* — nothing reconciles the views, no shared state is consulted. All four are outputs of one computation over one set of latents, so when the ball is struck every view shows it at that instant. **Consistency is structural rather than enforced.**

The cost of that elegance is rigidity. The network's inputs and outputs carry exactly four view streams and four action streams, and its cross-view attention was learned over that fixed set. **You cannot add a fifth player without retraining** — the same class of problem as asking a model trained to emit 4 channels to emit 5. The dataset is 4-view too.

MIRA has no memory mechanism, and the task hides it. Rocket League is one fixed symmetric arena, almost entirely in frame at all times — very little offscreen geometry to forget. Note also that **dynamics ≠ memory**: MIRA learned collisions and physics (how things move *now*), which is a different capability from recalling what was there before. Put the same architecture in a world where you walk into a building and back out and the gap appears immediately.

Generalise this: always ask **what the benchmark's environment lets the model skip**.

MultiGen — put the world outside the network

MultiGen does not try to make the neural rollout remember. It moves the world into an explicit data structure, so there is nothing to forget.

- **Memory** stores two things: a **static level map** as 2D **vertices and line segments** (walkable layout and walls), and **dynamic player poses** ($x, y, \text{yaw } \theta$). Symbolic and tiny — which is *why* levels can be authored as coarse 2D geometry and replayed reproducibly.
- **Observation** (a UNet with cross-attention, not a DiT) generates the next first-person frame from: the last **L** frames, the **next action**, and — the key trick — a **geometric signal produced by ray-tracing the 2D map from the current pose**: a 1-D depth vector across the field of view, converted to **disparity** (inverse depth) and concatenated with the context frames.

That ray-traced disparity channel is the whole idea. The model is *told* the wall layout in front of it every frame. It only ever has to **render** geometry, never **remember** it — which is the manufactured anchor that V2V got from the camera.

- **Dynamics** is a small transformer encoder over (current pose, action, geometry signal, intermediate UNet features) predicting an **incremental pose update**, applied with angle wrapping.
- **Cross-player visibility**: when rendering player A's frame, query memory for other players **visible from A's pose** and fold them into A's geometric conditioning. Nothing synchronises *between* generators — they all read one authoritative state, so they cannot disagree.
- **Scaling**: ~20 FPS using **one A100 per player**. Arbitrary N players — because the count lives in the *system*, not the *weights*. Elegant interface, linear hardware cost.

Matrix-Game 3.0 — learned retrieval memory

The middle path: memory is neither authored (MultiGen) nor absent (MIRA), but **retrieved**.

1. **Camera-aware selection.** Feeding all history is $O(N^2)$ -impossible; a fixed recent window forgets everything older. Instead, use the **current camera pose and field of view** to select past frames whose view **overlaps** the current one. Relevance is *geometric, not recency-based* — which is why it can recall something thousands of steps old while staying cheap.
2. **Injected into the same attention space.** Retrieved memory latents are concatenated with past-frame and noisy-current latents into **one joint self-attention operation**. No separate memory module — just more tokens in the same set. (This is the last row of Part I's mask×token-set table.)
3. **Relative Plücker encoding.** A Plücker coordinate represents a **ray** in 3D (direction + moment), so each pixel carries a ray rather than an (x, y). Encoding memory tokens by their rays **relative to the current camera** tells the model *where in space* each remembered pixel sits. Without it, retrieved frames are geometrically unregistered pixel soup; with it, the model can align a remembered wall to the wall in front of it.

Result: 720p at up to 40 FPS with a 5B model, minute-scale spatial consistency.

The world-model landscape

Project	Space	Backbone	Rollout scheme	Open?	Notable
DIAMOND (2024)	pixels	UNet + EDM/Heun	self-rollout, per-step MSE	code	RL world model — trains an agent <i>inside</i> it. The only one here that does
GameNGen (2024)	VAE latents	UNet (fine-tuned SD 1.4)	noise-augmented context	closed	DOOM at 20 FPS on one TPU, ~4-step DDIM. Origin of the genre
Oasis (2024)	latents	ViT autoencoder + DiT	diffusion forcing	weights (500M)	First open-weight playable DiT world model
MineWorld (2025)	discrete tokens	pure AR transformer	autoregressive	code	The <i>non</i> -diffusion branch
Matrix-Game 2.0 (2025)	3D causal VAE	DiT 1.8B (from a Wan2.1-family checkpoint)	Self-Forcing + DMD , 3 steps	code MIT	25 FPS on one H100 . Action modules injected per DiT block — the ControlNet pattern
Matrix-Game 3.0 (2026)	latents	DiT 5B ($\rightarrow 2 \times 14B$)	+ retrieval memory	code MIT	720p @ 40 FPS
MultiGen (2026)	—	UNet + cross-attn	external memory	unstated	Arbitrary N players; ~20 FPS per A100 per player
MIRA (2026)	RAE (frozen DINOv3-L/16)	DiT 5B + 600M codec	diffusion forcing	code Apache-2.0 + dataset	No weights released
minWM (2026)	latents	pluggable (Wan2.1-1.3B, MMDiT)	Causal Forcing / ++	code	Full-stack framework: data $\rightarrow$ camera-control finetune $\rightarrow$ AR training $\rightarrow$ distillation $\rightarrow$ low-latency inference
LingBot-World (2026)	latents	Wan2.2 DiT + FSDP	causal pretraining	code + weights	720p/60fps, hour-long, multi-user. Best option to actually play a SOTA model

Two lineages. (a) pixel / UNet / EDM — DIAMOND, GameNGen. (b) latent / DiT / diffusion-forcing — Oasis, MIRA, Matrix-Game, minWM, LingBot. (a) is the better thing to learn on; (b) is where the field is.

DIAMOND vs GameNGen — the deepest difference is purpose, not architecture. DIAMOND is a world model *for learning* (an agent is trained inside it, with reward and termination heads). GameNGen is a simulator *for playing* — no agent inside. Secondary differences: pixel vs VAE latent, from-scratch-and-tiny vs fine-tuned-SD-1.4, EDM vs SD schedule.

Three independent teams converged on diffusion forcing (Oasis, MIRA, minWM). When three groups solving the same problem from different directions land on the same mechanism, that is a core concept rather than a fashion.

Part VIII — Evaluation & metrics

Every claim in Parts IV, VI and VII is a quality claim, and quality claims are where this field is weakest. **The governing rule: every scalar metric hides at least one failure mode. Report a fidelity number and a diversity number, or you are not measuring.**

What each metric actually catches

Metric	Catches	Sharp edge
FID	Distribution distance; the headline number	Sample-count sensitive, backbone-dependent, and conflates fidelity with diversity
Inception Score (IS)	Quality × diversity from a classifier	Largely superseded; know why
sFID / KID	Spatial features; unbiased small-sample estimator	Fix specific FID weaknesses
Precision & Recall / Density & Coverage	Splits fidelity from diversity	The instrument that exposes mode collapse — from CFG scale, from distillation, from adversarial losses
CLIP score	Text–image alignment	Only meaningful for conditional models
LPIPS / PSNR / SSIM	Perceptual and pixel similarity	Your primary metrics when judging an autoencoder (Part I), not a generator

Tooling: `torch-fidelity` / `clean-fid` (FID/KID/IS done correctly), `torchmetrics` (LPIPS/SSIM/PSNR), `prdc` (precision/recall/density/coverage).

Validate by relative ordering, not absolute FID

Published benchmark numbers come from near-full training runs — EDM’s CIFAR-10 result was roughly **400 V100-hours per model**. On any realistic learning or iteration budget you will land in the **FID 10–30** range, which is entirely sufficient to see real differences.

So the gate is not “hit the published number.” The gates are:

1. Unit tests pass and an **overfit-one-batch** run still collapses to ~0 loss.
2. The **ordering is right** — EDM should beat DDPM badly at low NFE; DDIM should track DDPM at 1000 steps and win at 50.

Wrong ordering = a bug. Merely high absolute FID = you trained less. Those are completely different diagnoses and conflating them wastes days.

A related cost fact worth internalising: DDIM is a *sampler*, not a model. It reuses DDPM’s weights with no retraining. Comparing four “methods” often means training three networks, not four.

The three evaluations people skip

1. **The guidance sweep.** Plot FID *and* Recall against w . FID improves then flattens; Recall falls monotonically. Takes an afternoon and permanently inoculates you against single-metric reasoning.
2. **The drift curve** (Part VI) — quality vs rollout position, overlaid across techniques on identical seeds.
3. **The return-to-viewpoint / loop-closure test** — drive a closed loop (turn 360°, or leave a room and come back) and compare the frame at the *returned* pose against the frame originally captured there (PSNR/LPIPS, plus the blunt question “is it even the same room?”). Report **loop length vs similarity**. A model with no spatial memory scores fine at loop length 0 and collapses immediately after.

Running only one of tests 2 and 3 is how systems claim “long-horizon” while failing the other axis.
They measure different failures and neither implies the other.

Part IX — Quick reference

Model landscape

Model	Formulation	Notes
SD 1.5 / SDXL	Diffusion (discrete, ϵ -pred)	Terminal-SNR bug; use DPM++ 2M Karras
EDM / EDM2	Diffusion (σ -space, preconditioned)	Heun solver; EDM2 holds ImageNet FID records
SD3 / Flux	Rectified flow (= linear-path FM)	logit-normal t, dynamic shift, Euler
Wan 2.1 / 2.2	Flow matching	Apache-licensed; the base for many derivatives
HunyuanVideo, Mochi, LTX-Video	Flow matching	
Meta Movie Gen	Flow matching	Explicit in the paper
Sora / Veo	Undisclosed	DiT architecture is public; the objective is not

DiT is the architecture; flow matching is the training objective. Nearly everything is a DiT now; the objective is the part that varies — and that closed labs do not publish.

Decision tables

Which solver

Model	Solver	Steps
Flow matching	Euler	20–30
Flow matching, <15 steps	flow-adapted DPM / Heun	8–15
Diffusion	DPM-Solver++ 2M Karras	20–30
Distilled	none — the method’s scheduler	1–4

Should I distil?

Steps needed	Action
20–50	No. Base + good solver.
8–15	Try base + tuned solver + shift first.
4–8	Yes — DMD2 (quality) or LCM-LoRA (cheap)
1–2	Yes — DMD2 or LADD; accept diversity loss

Which quantisation

Bottleneck	Use
Memory-bound (LLM decode)	weight-only is enough
Compute-bound (diffusion)	fp8 weights AND activations
Never	fp8 on the VAE; int4 on images

Which compiler

Situation	Use
GPU serving in Python (~95 % of cases)	<code>torch.compile</code>
Frozen model, fixed shapes, need last 20 %	TensorRT (via <code>torch-tensorrt</code>)
No Python allowed / embedded	TensorRT
Leaving the PyTorch ecosystem	ONNX

Speedup ledger

Lever	Gain	Requires
Step distillation (25 → 1)	25×	weeks of training
Token reduction (VAE 4→8×	up to 16×	retraining
FlashAttention	2–4×	free — already in your framework
Guidance distillation	2×	training
Step reduction (50 → 25)	2×	nothing
<code>torch.compile</code>	2×	one line
fp8 (W+A)	1.5–2×	a few days
Hand-written kernels	1.3–2×	months
TensorRT	1.2×	days of export pain
CUDA graphs alone	1.04×	included in compile

Formulas

FLOW MATCHING PATH

$$x_t = (1-t) \cdot x_0 + t \cdot \varepsilon \quad \text{target} = \varepsilon - x_0$$

TIMESTEP (logit-normal)

$$t = \text{sigmoid}(\text{randn}() \cdot s + m) \quad \text{SD3: } m=0, s=1$$

RESOLUTION SHIFT

$$t' = \text{shift} \cdot t / (1 + (\text{shift}-1) \cdot t)$$

equivalently: $t'/(1-t') = \text{shift} \cdot t/(1-t)$

equivalently: $\lambda' = \lambda - 2 \cdot \log(\text{shift})$ ← a pure log-SNR translation

FLUX DYNAMIC SHIFT

$$m = (\text{max_shift} - \text{base_shift}) / (\text{max_seq} - \text{base_seq})$$
$$\mu = \text{image_seq_len} \cdot m + b; \quad \text{shift} = \exp(\mu)$$

256 tok → 1.65 1024 tok → 1.96 4096 tok → 3.16

LOG-SNR (linear path)

$$\lambda = 2 \cdot \log((1-t)/t) = -2 \cdot \text{logit}(t)$$

CFG

$$v = v_{\text{uncond}} + w \cdot (v_{\text{cond}} - v_{\text{uncond}}) \quad w: 3.5-5 \text{ (FM)}, 7-8 \text{ (SD)}$$

EULER ERROR

$$\text{error} \approx \frac{1}{2} \cdot h^2 \cdot a \quad h = \text{step size}, \quad a = \text{curvature}$$

zero if EITHER factor is zero

PROBABILITY-FLOW ODE

$$dx/dt = f(t) \cdot x - \frac{1}{2} g(t)^2 \cdot \nabla \log p(x, t) \quad \leftarrow \text{no random term}$$

ROOFLINE RIDGE (H100, bf16)

$$990 \text{ TFLOP/s} \div 3.35 \text{ TB/s} \approx 300 \text{ FLOP/byte}$$

below → memory-bound above → compute-bound

Glossary

Term	Meaning
adaLN	Adaptive layer norm — how DiTs inject timestep/condition. Source of activation outliers under fp8.
Arithmetic intensity	FLOPs $\div$ bytes moved. Compare to the ridge (~ 300 on H100 bf16).
CFG	Classifier-free guidance. Amplify the caption's contribution. Costs $2\times$.
Condition dropout	Blank the caption $\sim 10\%$ of training steps so one model serves as both conditional and unconditional.
CUDA graph	A recorded kernel sequence replayed with one call. Requires fixed shapes and addresses.
DiT	Diffusion Transformer — transformer denoiser over latent patches.
DMD2	Distribution Matching Distillation v2. Uses the teacher as a score <i>oracle</i> , not an answer key — hence can exceed it.
Exposure bias	Train on clean history, infer on own (erroneous) history. Cause of drift.
FlashAttention	Tiled attention with online softmax. Never materialises the $N\times N$ matrix. Exact. Converts attention from memory- to compute-bound.
History augmentation	Inject synthetic corruption into history during training so the model learns to correct rather than propagate. Cheap drift fix.
Kernel	One GPU program for one operation.
LADD	Latent Adversarial Diffusion Distillation. Discriminator in latent space using teacher features.
LCM	Latent Consistency Model. Trajectory distillation; LCM-LoRA is the cheap adapter form.
log-SNR (λ)	Log signal-to-noise ratio. The natural coordinate for noise schedules.
Mega kernel	Many fused ops in one launch, keeping intermediates on-chip.
NFE	Number of Function Evaluations — network forward passes. With CFG, $2\times$ the step count.
Online softmax	Running max + running sum, enabling incremental softmax. What makes FlashAttention possible.
Reflow	Retrain on your own model's (noise, image) pairs to make the coupling deterministic. Works; lost to distillation on cost.
Ridge point	The arithmetic intensity at which a GPU flips memory $\leftrightarrow$ compute bound.

Term	Meaning
RoPE	Rotary position embedding. Rotates q/k by angle $\propto$ position; absolute positions cancel, leaving relative. A function, not a table — hence extensible.
Self-forcing	Roll out the student's own generations during training. Expensive drift fix.
Shift	Reweights the noise schedule for resolution. A constant translation in log-SNR.
Shortcut distillation	Step-size-conditioned self-distillation. One model, $1..N$ steps, no fake-score net.
Stream (CUDA)	A queue. Within it, strictly ordered. PyTorch uses one by default.
Terminal SNR	Whether x_T is truly pure noise. Diffusion leaks $\sim 6\%$ ($\rightarrow$ no black/white images); flow matching cannot.
YaRN	Best-practice RoPE context extension. NTK-by-parts + attention temperature.
Zero terminal SNR	Diffusion's patch: rescale so $\bar{a}_T = 0$, plus v -prediction and trailing spacing.

Reading list

Foundations

- Sohl-Dickstein et al. 2015 — *Deep Unsupervised Learning using Nonequilibrium Thermodynamics* (diffusion, invented and then ignored for 5 years)
- Song & Ermon 2019 — *NCSN* (made it work at image scale)
- Ho et al. 2020 — *DDPM* (ϵ -parameterisation + simplified loss)
- Song et al. 2021 — *Score-Based Generative Modeling through SDEs* (the unification; the probability-flow ODE)
- Karras et al. 2022 — *Elucidating the Design Space (EDM)* (preconditioning; still the best-written paper in the field)
- Kingma & Gao 2023 — *Understanding Diffusion Objectives as the ELBO* (all objectives are one weighted integral over log-SNR)

Flow matching

- Lipman et al. 2022 — *Flow Matching for Generative Modeling*
- Liu et al. 2022 — *Rectified Flow* (and reflow)
- Albergo & Vanden-Eijnden 2022 — *Stochastic Interpolants*
- Esser et al. 2024 — *SD3* (logit-normal timesteps, resolution shift, the scaling ablation)

Solvers

- Song et al. 2020 — *DDIM*
- Lu et al. 2022 — *DPM-Solver / DPM-Solver++* (the exponential-integrator trick)
- Zhao et al. 2023 — *UniPC*

Distillation

- Salimans & Ho 2022 — *Progressive Distillation*
- Song et al. 2023 — *Consistency Models*
- Luo et al. 2023 — *LCM / LCM-LoRA*
- Yin et al. 2023/2024 — *DMD / DMD2*
- Sauer et al. 2024 — *ADD* (SDXL-Turbo) / *LADD* (SD3-Turbo)
- Frans et al. 2024 — *Shortcut Models*

Fixes worth knowing

- Lin et al. 2023 — *Common Diffusion Noise Schedules and Sample Steps are Flawed* (terminal SNR)
- Podell et al. 2023 — *SDXL* (size and crop conditioning)

Real-time video

- Chen et al. 2024 — *Diffusion Forcing*
- *CausVid* — bidirectional → causal + DMD distillation
- *Self-Forcing* — autoregressive rollout to fix exposure bias
- Decart — *MirageLSD* (history augmentation, shortcut distillation, custom CUDA mega-kernels)

Architecture & latent space

- Peebles & Xie 2022 — *DiT* (patch size is the compute knob; adaLN-Zero; the scaling curve)
- Esser et al. 2024 — *SD3 / MMDiT* (two-stream joint attention)
- Zheng, Ma, Tong, Xie 2025/26 — *Diffusion Transformers with Representation Autoencoders* (frozen DINO encoder + ViT decoder)
- Zhang et al. 2023 — *ControlNet* (zero-initialised clone of a frozen encoder)

World models

- Alonso et al. 2024 — *DIAMOND* (pixel-space EDM world model; agent trained inside)
- Valevski et al. 2024 — *GameNGen* (DOOM; noise-augmented context)
- Chen et al. 2024 — *Diffusion Forcing* (independent per-frame noise levels)
- Huang et al. 2025 — *Self-Forcing*
- thu-ml 2026 — *Causal Forcing* / ++
- Skywork 2025/26 — *Matrix-Game 2.0 / 3.0* (Self-Forcing+DMD; retrieval memory + Plücker)
- Po et al. 2026 — *MultiGen* (external memory; Memory/Observation/Dynamics)
- General Intuition & Kyutai 2026 — *MIRA* (representation autoencoder + diffusion forcing)

Systems

- Dao et al. 2022 / 2023 — *FlashAttention / FlashAttention-2*
- Shah et al. 2024 — *FlashAttention-3* (the Hopper rewrite)
- Williams et al. 2009 — *Roofline: An Insightful Visual Performance Model*

Ecosystem: when HuggingFace, when raw PyTorch

Verified from the actual dependency files of the projects in Part VII. The split is sharp, and it is a useful rule:

If you are...	You use	Evidence
Training from scratch (research, novel architecture)	Raw PyTorch	MIRA's <code>pyproject.toml</code> : <code>torch</code> , <code>einops</code> , <code>pydantic</code> , <code>hydra</code> , <code>wandb</code> , <code>torchmetrics</code> , <code>lpips</code> , <code>pytorch-fid</code> . No diffusers, no transformers, no accelerate. DIAMOND: same
Building on a pretrained foundation model	HF everywhere	Self-Forcing pins <code>diffusers==0.31.0</code> , <code>transformers>=4.49</code> , <code>accelerate>=1.1.1</code> — because it builds on Wan2.1. Matrix-Game 2.0 and minWM initialise from Wan-family checkpoints, so likewise
Anyone shipping weights or datasets	HF Hub , universally	Even MIRA — no <code>diffusers</code> , but <code>huggingface_hub</code> + <code>datasets</code> to distribute its dataset

Rule of thumb: from scratch → raw torch; on top of pretrained → HF. Learn `diffusers` as the *ecosystem* — how to consume pretrained components, read reference implementations, pull weights — not as the framework you write research inside.

The components you will actually reach for:

```
from diffusers import AutoencoderKL, UNet2DConditionModel, DiTTransformer2DModel, DDIMScheduler
from transformers import CLIPTextModel, T5EncoderModel, AutoModel, AutoImageProcessor

vae = AutoencoderKL.from_pretrained("stabilityai/sd-vae-ft-mse")
UNET = UNet2DConditionModel.from_pretrained("runwayml/stable-diffusion-v1-5", subfolder="UNET")
txt = CLIPTextModel.from_pretrained("openai/clip-vit-large-patch14")
dino = AutoModel.from_pretrained("facebook/dinov2-large") # the RAE encoder, Part I
```

Habits: `subfolder=` extracts one component from a full pipeline repo; `.requires_grad_(False)` + `bf16` for anything frozen; **apply the VAE scaling factor** (Part I); give DINO *its* normalisation, not SD's.

The cross-implementation unit test. Load SD's UNet, discard the pipeline, and run **your own** sampler against their weights. Good images ⇒ both your sampler and your understanding are correct. This is the fastest way to validate a from-scratch implementation.

Build the DiT yourself before reading theirs. Patchification, adaLN-Zero and RoPE cannot be learned by calling a pipeline. Then diff yours against `diffusers`' implementation line by line — every difference is either a bug in yours or a production concern you had not met.

Cost reference

⚠️ **Approximate GCP on-demand, mid-2026 — verify before committing.** $T4 \approx \$0.35/\text{hr} \cdot L4 \approx \$0.70/\text{hr} \cdot A100-40GB \approx \$3.67/\text{hr} \cdot A100-80GB \approx \$5.07/\text{hr} \cdot H100 \approx \$9-11.50/\text{hr}$. **Spot cuts 60–91 %** — and checkpoint/resume discipline is what makes Spot safe.

Thing	Realistic cost
CIFAR-10 scale model, “good enough to compare”	hours on an L4 — \$5–15 each
Published-FID reproduction (EDM CIFAR-10)	~400 V100-hours per model — don’t
Class-conditional ImageNet-64 from scratch	tens of GPUs for a week+ — \$2–10k . Load open weights instead
DiT-XL/2 on ImageNet-256	7M steps on 8×A100 — out of reach
DIAMOND CS:GO reproduction	~12 days on one RTX 4090 ($\approx \$100-200$ rented) — cheap, high impact
MultiGen-style serving	one A100 per player at 20 FPS
MIRA at full scale	needs a B200 for 20 fps <i>because it is not distilled</i>
Matrix-Game 2.0	25 FPS on one H100 <i>because it is</i> (3 steps)

The pattern in the last three rows is the whole economic argument for Part IV. Distillation is not a nicety; it is the difference between a B200 and an H100 per stream.

Part X — Project notes

Specific to this repository. Everything above is general; this Part is not.

Findings in the vendored DIAMOND code

Location	Finding
src/models/diffusion/inner_model.py:38	It is a UNet
inner_model.py:42	Zero-inits the output conv — the zero-init pattern from Part I
inner_model.py:46	Past frames concatenated as channels (<code>torch.cat((obs, noisy_next_obs), dim=1)</code>)
config/agent/default.yaml:14,24	<code>attn_depths: [0,0,0,0]</code> — attention is off at every down/up level. The only attention is the hardcoded mid block (<code>blocks.py:216</code>). Effectively a pure-conv UNet with one attention bottleneck
denoiser.py:61–71	EDM preconditioning (<code>c_in / c_skip / c_out / c_noise</code>) plus <code>sigma_offset_noise</code>
—	Action and noise conditioning enter via AdaGroupNorm (FiLM-style), not cross-attention or tokens

⚠ **The important one: `denoiser.py:100–119` — DIAMOND is *not* teacher forcing.** The training loop iterates the sequence and writes each *denoised prediction* back into the conditioning buffer:

```
for i in range(seq_length):
    obs = all_obs[:, i : n + i]          # conditioning window
    ...
    denoised = self.wrap_model_output(noisy_next_obs, model_output, cs)
    all_obs[:, n + i] = denoised          # ← its OWN prediction, fed forward
```

So later steps condition on the model’s own generations — **self-rollout**, an early member of the Part VI forcing family. What it lacks versus Self-Forcing: a distribution-matching loss (it uses per-step MSE), a KV cache (conv UNet — nothing to cache), and a few-step student. **Diffing those two training loops is the single most instructive exercise available in this repo.**

Cheap upgrades to DIAMOND, ranked by effort:

1. **History augmentation** (noise-augmented context) — a handful of lines; gives a measurable drift-curve delta.
2. **Per-frame independent σ** (diffusion forcing) — a small change to `apply_noise` / `compute_conditioners`; sample `[B, T]` instead of `[B]`.
3. **Distribution-matching loss** instead of per-step MSE — the real jump, much more work, and the point at which moving to a DiT makes more sense.

Items 1–2 are genuinely cheap and produce a publishable-shaped ablation on a model you already trained.

Open item in our own package

`src/diffusion/train/trainer.py:44` and `:136` construct `UNet(...)` **directly**, and `:136` re-lists all five model fields by hand to build the EMA evaluation copy. Needs a `build_model(cfg.model)` registry plus `ModelConfig.name`, so that adding a DiT is a config line rather than a refactor — and it deletes the duplication as a side effect. Samplers for `ddpm/ddim/edm/flow-matching` are already scaffolded under `core/samplers/`.

Open questions

- **Does MIRA use a flow-matching/rectified-flow objective, or ϵ/v -prediction?** Not stated in the repo README. Most 2025–26 video DiTs use flow matching, but unconfirmed — check the technical report.
- **Is MultiGen’s code released?** Not stated on the project page or in the paper.
- **MIRA’s exact decoder configuration** — RAE-family decoders are ViTs (confirmed for the RAE paper itself), but MIRA’s specifics are not in the README.
- **Matrix-Game 3.0 training compute** — not reported in what we read.
- Survey listings mention **AlayaWorld** and **StableWorld** (2026), surfaced only via a secondary summary. Verify before relying on them.

Corrections log

Claims that were wrong at some point in this project’s notes, and their corrections. Recorded so the wrong version does not get re-learned.

Claim	Correction
“DIAMOND does naive frame-stacking / teacher forcing”	Wrong. <code>denoiser.py:100–119</code> does self-rollout — it conditions on its own denoised predictions
“MIRA achieves stability without anti-drift tricks”	Imprecise. MIRA does use diffusion forcing; the claim is no <i>additional</i> specialised machinery, with the codec credited as the main contributor
“Hold CIFAR-10 fixed for the DiT experiment”	Fine for a controlled A/B, but CIFAR is too small for any latent experiment — an $8\times$ VAE gives a 4×4 latent. Move to $\geq 64\times 64$
“Validate against published FID targets”	Those cost ~ 400 V100-hours per model. Validate by relative ordering instead
“CFG, LoRA/PEFT” presented together as if related	Unrelated. CFG is inference-time guidance; LoRA is parameter-efficient fine-tuning
“ControlNet $\approx$ guided inference”	No. ControlNet is <i>trained</i> (new parameters, paired data); guidance is <i>inference-time</i> score reweighting
Project names “GameMatrix / MiniWM / LongBot”	Actual projects: Matrix-Game, minWM, LingBot-World
Reflow presented as a live few-step technique	Superseded by DMD2 / sCM / MeanFlow

The whole document in twelve lines

1. **One idea underneath everything:** blur the data into a Gaussian, learn $E[x_0 | x_t]$, walk back down. Diffusion and flow matching are two coordinate systems on that idea.
2. **Flow matching wins on bug-resistance**, not on straightness. Simpler parameterisation, correct terminal condition, and it is where the recipes live.
3. **Error** = $\frac{1}{2} \cdot h^2 \cdot \text{curvature}$. Every solver, path, and schedule decision attacks one of those two factors.
4. **Curvature has two sources**. Flow matching deletes the schedule arc; DPM-Solver++ integrates it exactly. **Neither touches the Bayes-averaging curvature — which is why distillation exists.**
5. **Solvers are always used**. Euler for flow matching at 20–30 steps; DPM-Solver++ 2M for diffusion. Schedule and step count matter more than solver choice.
6. **After distillation there is no solver**. The model outputs an image, not a direction. Predict x_0 , re-noise, repeat.
7. **Resolution shift is the most common serious training bug**. High resolution makes the task too easy, so the model never learns to invent structure.
8. **Diffusion is compute-bound**, unlike LLM decode. So fp8 must cover weights *and* activations; weight-only buys you nothing.
9. **FlashAttention does not reduce FLOPs**. It removes the traffic that was starving them — converting attention from memory-bound to compute-bound.
10. **Optimise in order: fewer steps (25×) → fewer tokens → compile + fp8 (3–4×) → kernels (1.3–2×)**. Never invert. If you have not cut step count, do not write a CUDA kernel.
11. **Real-time needs a causal architecture, not just a fast one**. A bidirectional model cannot stream at any speed. And video-to-video anchoring makes drift bounded, which is why restyling ships before free generation.
12. **Latency is the hull**. Fine-tuning changes weight values, never the shape. But you *can* design your own shape and distil the knowledge into it — keep the teacher’s VAE, redesign the transformer.